PCG Arena: Evaluating and Improving Super Mario Bros Level Generators Through Player Preferences

(PCG Arena: ewaluacja i ulepszanie
generatorów poziomów do gry Super Mario Bros
na podstawie preferencji graczy)

Antoni Czapski

Praca magisterska

Promotor: dr Jakub Kowalski

Uniwersytet Wrocławski
Wydział Matematyki i Informatyki
Instytut Informatyki

September 2026

Abstract

Evaluating procedural content generators (PCGs) for video games poses significant methodological challenges, as player enjoyment is subjective and difficult to quantify through automated metrics alone. This thesis presents PCG Arena, a web-based platform for blind A/B testing of Super Mario Bros level generators using human preference data. Super Mario Bros has been regularly used as a benchmark for PCG level generation research since the first Mario AI Competition in 2009 and is a natural domain for systematic generator comparison. On the platform, players play two procedurally generated levels side-by-side in their browser, vote for their preferred level, and optionally tag gameplay characteristics. Votes are aggregated with the Glicko-2 rating system to produce uncertainty-aware rankings and confidence intervals.

Using the collected data, we conduct an exploratory analysis of generator preference rankings, static expressive range, gameplay trajectories, player consistency, and tag semantics. The results show that PCG Arena produces a reliable leaderboard while also diagnosing why specific generators win or fail. They further show that automated structural metrics cannot substitute for the human ranking, which motivates training a generator directly on player preferences.

Finally, we present MarioDPO, a level generator fine-tuned with Direct Preference Optimization that achieves the highest win rate among the tested procedural generators and approaches the hand-crafted Nintendo baseline.

Ewaluacja generatorów treści proceduralnych (PCG) dla gier wideo stanowi istotne wyzwanie metodologiczne, ponieważ przyjemność gracza jest z natury subiektywna i trudna do zmierzenia za pomocą automatycznych metryk. Niniejsza praca przedstawia PCG Arenę, platformę internetową do ślepego testowania A/B generatorów poziomów do gry Super Mario Bros poprzez dane o preferencjach graczy. Super Mario Bros jest regularnie wykorzystywane jako benchmark w badaniach nad proceduralnym generowaniem poziomów od pierwszych zawodów Mario AI Competition w 2009 roku, dlatego stanowi naturalną dziedzinę do systematycznego porównywania generatorów. Platforma umożliwia graczom rozgrywanie dwóch proceduralnie generowanych poziomów obok siebie w przeglądarce, głosowanie na preferowany poziom oraz opcjonalne tagowanie cech rozgrywki. Głosy są agregowane za pomocą systemu ratingowego Glicko-2, który wyznacza rankingi wraz z przedziałami ufności.

Na podstawie zebranych danych przeprowadzono analizę eksploracyjną obejmującą ranking preferencji generatorów, statyczny zakres ekspresyjny poziomów, trajektorie rozgrywki, spójność ocen anonimowych graczy oraz semantykę tagów. Wyniki pokazują, że platforma pozwala tworzyć ranking generatorów oraz diagnozować ich typowe tryby porażki. Analiza wykazuje ponadto, że automatyczne metryki strukturalne nie są w stanie zastąpić rankingu opartego na ludzkich preferencjach, co stanowi motywację do trenowania generatora bezpośrednio na głosach graczy.

W ostatniej części pracy przedstawiono MarioDPO, generator poziomów dostrojony metodą Direct Preference Optimization, który osiąga najwyższy współczynnik zwycięstw spośród badanych generatorów i zbliża się wynikiem do oryginalnych poziomów Nintendo.

Contents

1 Introduction **9**

1.1 Procedural Content Generation in Video Games 9

1.2 The Evaluation Problem 10

1.3 Research Questions and Contributions 11

1.4 Thesis Structure 12

2 Background and Related Work **13**

2.1 Super Mario Bros 13

2.2 Procedural Content Generation 14

2.2.1 Search-Based Approaches 14

2.2.2 Grammar-Based and Rule-Based Approaches 15

2.2.3 Machine Learning-Based Generation (PCGML) 15

2.3 Super Mario Bros as a PCG Benchmark 16

2.3.1 The Mario AI Framework 16

2.3.2 Mario Level Generators in the Literature 17

2.4 Evaluation Methods for Procedural Content 18

2.4.1 Automated Metrics 18

2.4.2 Agent-Based Testing 19

2.4.3 Human Evaluation Studies 19

2.4.4 Comparison of Approaches and Their Limitations 20

2.5 Rating Systems for Pairwise Comparisons 22

2.5.1 The Elo Rating System 22

2.5.2 The Glicko-2 Rating System 22

2.5.3 Arena-Style Evaluation Platforms . . . 23
2.6 Preference Learning and Alignment . . . 23
2.6.1 Reinforcement Learning from Human Feedback . . . 23
2.6.2 Direct Preference Optimization . . . 24

3 PCG Arena — System Design 25
3.1 Requirements and Design Goals . . . 25
3.2 System Architecture . . . 26
3.2.1 Frontend . . . 26
3.2.2 Backend . . . 26
3.2.3 Database . . . 27
3.3 Game Engine . . . 27
3.3.1 TypeScript Port . . . 28
3.3.2 Level Format . . . 29
3.3.3 Level Validation . . . 30
3.4 Battle and Voting System . . . 31
3.4.1 Battle Flow . . . 31
3.4.2 Tagging System . . . 33
3.5 Matchmaking: AGIS Algorithm . . . 34
3.5.1 Problem Definition . . . 34
3.5.2 Stage 1: First Generator Selection . . . 34
3.5.3 Stage 2: Second Generator Selection . . . 35
3.5.4 Parameters . . . 35
3.6 Glicko-2 Rating System . . . 36
3.6.1 Rating Update Procedure . . . 36
3.6.2 Configuration . . . 37
3.7 Builder Profile and Generator Submission . . . 40
3.8 Authentication . . . 42
3.9 Deployment and Infrastructure . . . 43

4 User Study — Measuring Player Preference 45

4.1 Study Protocol and Dataset 45
4.2 Generator Preference Ranking 48
4.3 Player Engagement and Preference Consistency 49
4.4 What Static Metrics Can and Cannot Tell Us 52
4.5 Why Levels Lose: Trajectories and Tags 55
4.6 The Judge Function 58
4.7 Summary: A Reliable but Sparse Signal 59

5 MarioDPO — Preference-Aligned Level Generation 61
5.1 Motivation: From Sparse Votes to a Training Signal 61
5.2 Level Representation and Base Model 62
5.2.1 Column-Major Tokenisation 62
5.2.2 Base Model 63
5.3 Preference Dataset Construction 63
5.4 DPO Fine-Tuning and Inference-Time Filtering 64
5.5 Results 65
5.6 Discussion 66

6 Conclusions and Future Work 69
6.1 Summary of Contributions 69
6.2 Limitations 70
6.3 Future Directions 71
6.4 Closing Remarks 71

Chapter 1

Introduction

1.1 Procedural Content Generation in Video Games

Procedural Content Generation (PCG) is the algorithmic creation of game content (such as levels, maps, characters, items, and narratives) with limited or indirect human input [32, 24, 31]. PCG has a long commercial history: *Rogue* (1980) and *Elite* (1984) used algorithmic generation to overcome memory constraints, and modern titles such as *Minecraft*, *No Man's Sky*, and *Spelunky* use procedural generation to create replayable, large-scale worlds. Academic research on PCG has grown steadily since the mid-2000s and spans constructive rules, evolutionary search, formal grammars, neural networks, and reinforcement learning.

Among the game domains studied, Super Mario Bros (SMB) has accumulated the largest body of comparative work in level generation. Its 2D tile-grid format defines a tractable design space, its physics are well understood, and the open-source Mario AI Framework [10] provides a standardised environment for both automated and human evaluation. Over 15 years of research has produced dozens of Mario level generators across most major PCG paradigms, yet evaluating and comparing them remains an open challenge.

Figure 1.1 shows the original game and its tile-based level geometry.

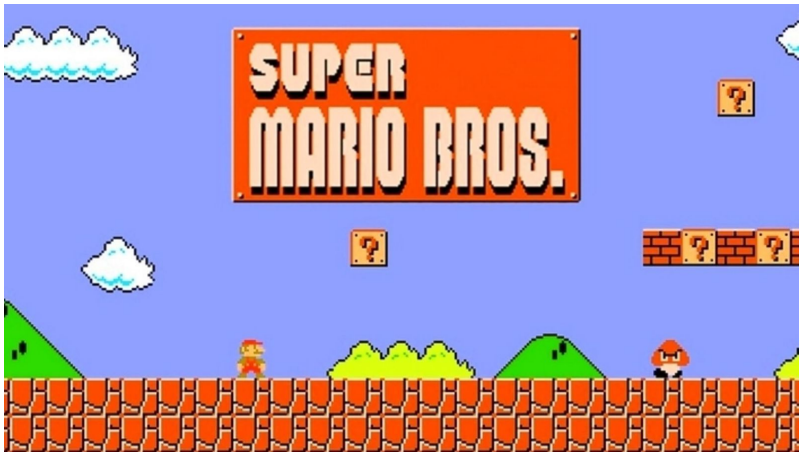


Figure 1.1: Screenshot of the original *Super Mario Bros* (Nintendo, 1985). The player controls Mario through side-scrolling levels with platforms, enemies, power-ups, and coins. The 2D tile-grid structure of these levels defines the design space studied in this thesis.

1.2 The Evaluation Problem

Evaluating procedural content generators is difficult because the ultimate quality criterion, player enjoyment, is subjective and expensive to measure. The PCG literature has developed three main families of evaluation methods, each with clear limitations.

Automated structural metrics, such as linearity, leniency, and density, can be computed directly from level tilemaps and are widely used to characterise generator output [25, 9]. However, Mariño et al. [12] demonstrated that these metrics only weakly correlate with human-rated difficulty and enjoyment, concluding that "current computational metrics should not be used in lieu of user studies." Multiple incompatible definitions of the same metric (at least three formulations of leniency exist in the literature) also complicate cross-study comparisons.

Agent-based testing uses AI controllers, typically the A* agent from the 2009 Mario AI Championship, as player proxies. Agent-derived features such as jump count or completion fraction provide scalable difficulty estimates [33], but superhuman agents can solve levels that human players cannot. Difficulty estimates also depend on the specific agent policy, which introduces a systematic bias that is rarely quantified.

Human evaluation studies remain the gold standard, but they are expensive and usually small-scale: controlled lab studies use 15–37 participants [21, 12], while larger web-based studies sacrifice experimental control [16]. Small samples cannot control for differences in gaming experience, player expectations, or learning effects across trials. Existing studies were also conducted as one-time events, often

years apart, and the largest web-based studies predate modern ML-based generators (GANs, LLMs, and diffusion models). Many studies also reveal to participants which algorithm produced each level, even though knowing the source generator can bias player judgments.

This persistent gap between scalable-but-inaccurate automated metrics and valid-but-expensive human studies motivates the work presented in this thesis.

1.3 Research Questions and Contributions

This thesis addresses the PCG evaluation gap through the design, deployment, and analysis of *PCG Arena*, a web-based platform for blind A/B testing of Mario level generators. We investigate three research questions:

RQ1 (Measurement)
Can a persistent, blind, web-based arena produce a reliable human-preference ranking of Super Mario Bros level generators? (Sections 4.2–4.3)

RQ2 (Metrics)
Can structural properties of levels explain human ranking? (Sections 4.4–4.6)

RQ3 (Generation)
Can the collected preference data be used to train a level generator that surpasses existing PCG methods and approaches human-authored quality? (Chapter 5)

The thesis makes four contributions:

1. **PCG Arena**, a persistent, open-access web platform for blind A/B testing of Super Mario Bros level generators with browser-based gameplay, Glicko-2 uncertainty-aware rankings, and a custom matchmaking algorithm (AGIS).
2. **A user study** of 1,000 exported pairwise votes from 77 anonymous player IDs across 14 research generators, including 1,968 played side records, gameplay trajectories, and descriptive tags.
3. **An exploratory analysis pipeline** that connects blind preference votes to static expressive-range metrics, trajectory patterns, player-level preference profiles, and tag-based failure-mode diagnostics.
4. **MarioDPO**, a level generator fine-tuned with Direct Preference Optimisation on arena votes that achieves a preference score of 0.832, close to the hand-crafted Nintendo baseline.

The platform source code is available at `https://github.com/antoniczapski/pcg-arena`. The MarioDPO implementation, analysis scripts, and thesis sources are available at `https://github.com/antoniczapski/mario-dpo`.

1.4 Thesis Structure

The remainder of this thesis is organised as follows. Chapter 2 surveys background and related work on procedural content generation in the context of Mario level generation, evaluation methodology, rating systems, and preference learning. Chapter 3 describes the design and implementation of the PCG Arena platform. Chapter 4 presents the user study: the study protocol and collected dataset, the generator preference ranking and player-consistency analysis (RQ1), and an assessment of whether static structure, gameplay behaviour, and tags can substitute for the human ranking (RQ2). Chapter 5 uses these results (RQ3): it describes the level representation and base model, the construction of a preference dataset from human votes and judge-labelled synthetic pairs, DPO fine-tuning, and the evaluation of the resulting MarioDPO generator. Chapter 6 summarises contributions, discusses limitations, and outlines future directions.

Chapter 2

Background and Related Work

2.1 Super Mario Bros

Super Mario Bros (SMB), released by Nintendo in 1985, is a side-scrolling platform game in which the player controls Mario through a sequence of levels. Each level is composed of a 2D tile grid, typically 16 tiles tall and 150–210 tiles wide, scrolling horizontally from left to right. The game's design vocabulary includes several categories of elements:

- **Terrain and platforms.** Solid ground blocks (`X`), brick blocks (`S`) that can be broken by hitting from below, pyramid blocks (`#`), and jump-through platforms (`%`) form the physical structure of each level.
- **Interactive blocks.** Question blocks may contain coins or power-ups: a Super Mushroom (which transforms Small Mario into Super Mario, allowing him to break bricks and survive one extra hit), a Fire Flower (enabling ranged attacks), or a 1-Up Mushroom (extra life). Invisible blocks provide hidden rewards.
- **Enemies.** Goombas are defeated by jumping on them; Koopa Troopas retract into their shells, which can then be kicked as projectiles; Spikies (Spinies) cannot be stomped; and Bullet Bills are fired from cannons. Winged variants of each enemy add vertical movement.
- **Collectibles.** Coins are scattered across levels as floating items or hidden inside blocks. Collecting 100 coins grants an extra life.
- **Pipes.** Decorative or functional vertical structures, some of which spawn Piranha Plant enemies.
- **Level boundaries.** Each level begins at a Mario spawn point (`M`) and ends at a flag pole (`F`).

The game's physics (gravity, inertia, jump arcs, and collision detection) are deterministic and well-characterised. These properties make SMB a practical benchmark for PCG research: the tile-grid format defines a discrete design space, the mix of terrain, enemies, and power-ups provides a well-understood design vocabulary,

and the game's familiarity means that most experimental subjects already know the controls and goals (Figure 1.1).

2.2 Procedural Content Generation

Procedural Content Generation (PCG) is the algorithmic creation of game content with limited or indirect human input [32, 24]. Following the taxonomies of Togelius et al. [32], Shaker et al. [24], and Summerville et al. [31], we distinguish five broad generation paradigms:

Constructive / Rule-based
Sequentially place content according to hand-authored rules or grammars. Fast and reliable, though limited in expressiveness.

Search-based (SBPCG)
Frame generation as optimisation over a fitness function, typically using evolutionary algorithms. Flexible, though computationally expensive.

Grammar / Pattern-based
Use formal grammars, design patterns, or n-grams to encode structural idioms. Captures human design style, provided a suitable pattern library is available.

Machine Learning (PCGML)
Train neural networks on existing human-designed content. Data-driven and expressive, though outputs do not guarantee diversity or playability.

Reinforcement Learning (PCGRL)
Frame level design as a Markov Decision Process and train an agent to construct content. Unlike PCGML, it requires no corpus of existing levels and relies instead on a reward function that encodes the desired content properties.

These paradigms are not mutually exclusive, and many modern systems are hybrids. For instance, MarioGAN [33] combines a learned GAN with CMA-ES search in its latent space. A related distinction separates *offline* generation (content precomputed before the game ships) from *online* generation (content created at runtime, potentially adapting to the player). Online generation supports personalisation under stricter runtime budgets, as studied in experience-driven PCG [34]. A useful property of any generator is *controllability*: the ability to accept parameters describing desired content features and produce matching levels.

2.2.1 Search-Based Approaches

Search-Based PCG (SBPCG) frames content generation as an optimisation problem [32]. A *fitness function* encodes the desired properties of the content (e.g.,

playability, target difficulty, aesthetic criteria), and an evolutionary algorithm (typically a genetic algorithm or CMA-ES) searches a space of candidate solutions to maximise it.

Fitness functions give designers explicit, modular control: playability constraints, difficulty targets, and diversity objectives can be combined into a single optimisation criterion. However, SBPCG has two well-known limitations. First, each fitness evaluation may require an expensive simulation (e.g., running an A* agent through a candidate level). Second, designing good fitness functions is itself a difficult problem that mirrors the broader evaluation challenge motivating this thesis.

The Mario AI Level Generation Competition [21] included several search-based entries. More recent work has applied CMA-ES to the latent space of trained generative models [33] to combine search-based and ML-based generation.

2.2.2 Grammar-Based and Rule-Based Approaches

Constructive generators build content sequentially according to hand-authored rules, typically working left-to-right through a level. The Notch generator shipped with *Infinite Mario Bros* [10] is the canonical example: it iterates through section templates (straight, hill, tubes, gaps, cannons) with fixed selection probabilities and basic playability checks. Constructive methods are fast and guarantee structural validity, but their expressive range is narrow.

Grammar-based approaches extend this idea by encoding design rules in a formal grammar. Shaker et al. [23] used *Grammatical Evolution* (GE) to map integer genotypes to level phenotypes via context-free grammar productions, combining the interpretability of grammars with evolutionary search. Smith and Whitehead [25] introduced *Launchpad*, a rhythm-based generator that maps player actions to level geometry via a design grammar. The same paper introduced *Expressive Range Analysis* (ERA), which generates a large sample of levels, computes structural metrics, and visualises their joint distribution; it has since become the standard method for characterising PCG generators.

2.2.3 Machine Learning-Based Generation (PCGML)

Machine learning approaches to PCG (PCGML) train neural networks on existing content to learn design patterns from data [31]. Several model families have been applied to Mario level generation.

Sequence models. Summerville and Mateas [29] trained a 3-layer LSTM to generate levels column-by-column using a "snaking" tokenisation that preserves vertical adjacency, and co-generating levels with A*-agent play traces reached 97% playability.

Generative adversarial networks. MarioGAN [33] trains a Deep Convolutional GAN on sliding-window segments from a single original SMB level. CMA-ES then searches the GAN's latent space to optimise fitness functions based on tile distributions or agent-derived playability. This work established the **Latent Variable Evolution** (LVE) paradigm.

Large language models. MarioGPT [28] fine-tunes DistilGPT-2 on column-tokenised Mario levels with text-prompt conditioning (e.g., "many pipes, many enemies, low elevation"), enabling the first natural-language-controllable level generator. 88% of generated levels are playable without post-processing.

Diffusion models. MarioDiffusion [19] uses a text-conditioned UNet denoiser to generate 16×16-tile scenes from natural-language captions, representing the latest generation of multimodal PCG.

Reinforcement learning. PCGRL [11] formulates level design as an MDP and trains an RL agent to edit tile grids, bypassing the need for training data entirely.

2.3 Super Mario Bros as a PCG Benchmark

Several practical factors explain why Super Mario Bros recurs throughout the PCG literature: its 2D tile-grid format is compact, its physics introduce no new mechanics after the first level (isolating level design as the variable of interest), most players are already familiar with the game's controls and goals, and the open-source Mario AI Framework provides a standardised evaluation environment.

2.3.1 The Mario AI Framework

The Mario AI Framework [10] is an open-source Java implementation based on *Infinite Mario Bros*, a public-domain clone of Nintendo's Super Mario Bros created by Markus Persson in 2008. It was the basis for three tracks of the Mario AI Championship (2009–2012):

1. **Gameplay Track**: build a controller that plays levels as well as possible.
2. **Learning Track**: build a controller that learns to play.
3. **Level Generation Track**: build software that generates levels for human players.

The Level Generation Track [21] was the first academic PCG competition. Generators received gameplay metrics from a test level and had 60 seconds to produce a personalised level. Six entries competed; evaluation used pairwise forced-choice ("which was more fun?") with 15 human judges. This competition established several norms that influenced subsequent research: pairwise comparison as the preferred human evaluation protocol, player telemetry as input to adaptive generators, and the

A* agent as a *de facto* playability oracle.

Levels are stored as 2D character grids (ASCII tilemaps), where each character encodes a functional tile type. The Video Game Level Corpus (VGLC) [29] standardised this encoding across all 32 original SMB levels and has been widely used as training data for neural generators.

2.3.2 Mario Level Generators in the Literature

The PCG Arena platform hosts 14 generators spanning all major paradigm families, selected to benchmark a broad cross-section of established PCG approaches: one human-authored baseline, four constructive generators, one chunk-based generator, four search-based generators, three neural generators, and MarioDPO, the preference-aligned generator introduced in this thesis. Table 2.1 provides a comparative overview.

An important caveat is that these generators do not all target the same design vocabulary. Several reduce the problem space substantially, for instance by restricting themselves to one or two enemy types (typically Goombas and Green Koopas) or omitting power-ups, pipes with Piranha Plants, or Bullet Bill cannons. This difference affects direct comparison: a generator that uses only a subset of SMB's design elements may score well on some structural metrics while remaining less expressive than the original Nintendo levels.

Human-authored baseline. The 15 original SMB levels from Nintendo (1985), transcribed into the Mario AI Framework's format as a reference against which algorithmic generators are compared.

Constructive generators. Four generators build levels left-to-right using probabilistic placement: the *Notch* generator shipped with Infinite Mario Bros, its *Parameterized* variant (NotchParam) exposing six tunable difficulty and style parameters [21], a *Randomized* variant (NotchParamRand) that randomly samples parameters per level, and *Hopper*, which alternates generated and pre-designed segments with adaptive probability adjustment.

Chunk-based assembly. *Occupancy-Regulated Extension* (ORE) [13] builds levels by iteratively attaching hand-authored chunks at compatible anchor points.

Search-based generators. The *Grammatical Evolution* (GE) generator [23] evolves level-construction programs via a context-free grammar. Three *Pattern-based* generators [5] represent levels as sequences of micro-patterns extracted from original SMB levels and use evolutionary search with different fitness functions: pattern count (maximising motif frequency), pattern occurrence (maximising motif diversity), and weighted pattern count (rewarding rare motifs).

Neural generators. *MarioGAN* [33] combines a DCGAN with CMA-ES

latent-space search. *MarioGPT* [28] uses a fine-tuned DistilGPT-2 with text-prompt control; *MarioDiffusion* [19] generates tile scenes from natural-language captions via a diffusion model; and *MarioDPO*, introduced in Chapter 5, fine-tunes a GPT-2-based generator with Direct Preference Optimisation on PCG Arena votes.

Generator	**Paradigm**	**Key technique**	**Control**	**Year**
Original SMB1	Human	Manual design	—	1985
Notch	Constructive	Probabilistic L-to-R	None	2008
NotchParam	Constructive	Probabilistic + 6 params	Param. knobs	2011
NotchParamRand	Constructive	Probabilistic + random	Random	2011
Hopper	Constructive	Adaptive probabilistic	Adaptive	2010
ORE	Chunk-based	Anchor-point assembly	None	2010
GE	Search	Grammatical evolution	Via grammar	2012
Pattern Count	Search	Evolutionary (pattern freq.)	Fitness wts.	2014
Pattern Occurrence	Search	Evolutionary (unique patt.)	Fitness wts.	2014
Pattern Wt. Count	Search	Evolutionary (rarity-wt.)	Fitness wts.	2014
MarioGAN	ML (GAN)	DCGAN + CMA-ES latent	Fitness fn.	2018
MarioGPT	ML (LLM)	Fine-tuned DistilGPT-2	Text prompts	2023
MarioDiffusion	ML (Diff.)	Text-conditioned UNet	Text captions	2025
MarioDPO	ML + DPO	GPT-2 + pref. alignment	Text + DPO	2026

Table 2.1: Generators included in PCG Arena. All except MarioDPO (this thesis) are previously published or reference implementations.

By including generators from all paradigm families and spanning nearly four decades of game content creation, PCG Arena tests whether human preference data can discriminate generator quality more reliably than automated metrics.

2.4 Evaluation Methods for Procedural Content

Evaluation is the central methodological challenge in PCG research and the primary motivation for PCG Arena. This section surveys the main families of evaluation methods used in the Mario PCG literature.

2.4.1 Automated Metrics

The most widely used evaluation approach computes *static structural metrics* directly from the tile grid, without simulating gameplay.

Linearity measures how closely a level's vertical profile follows a straight line. Smith and Whitehead [25] defined it as the mean absolute residual (the average vertical distance between platform midpoints and the best-fit regression line), so lower values indicate a more linear level. Later works [9, 30] instead use R^2 (the coefficient of determination, measuring the proportion of variance in the data explained by the

regression), which inverts the scale so that higher R^2 means more linear. Because the two formulations are not directly comparable, complicating cross-paper analyses.

Leniency approximates difficulty through weighted sums of game objects. In its original form [25], it is computed as $L = (w_p \cdot n_p + w_g \cdot n_g + w_e \cdot n_e)/W$, where n_p is the number of power-ups, n_g the number of gaps, n_e the number of enemies, W the level width in tiles, and the weights $w_p > 0$, $w_g, w_e < 0$ reward forgiving content and penalise hazards. At least three incompatible formulations exist in the literature, using different tile weights and normalisation schemes [25, 23, 12], and Mariño et al. [12] showed that leniency only weakly correlates with human-rated difficulty.

Compression distance (gzip-based Normalised Compression Distance) quantifies structural similarity between level pairs and is used as a diversity proxy [23, 9].

Expressive Range Analysis (ERA), introduced by Smith and Whitehead [25], generates a large sample of levels, computes two metrics per level (classically linearity vs. leniency), and plots their joint distribution as a 2D histogram. ERA has been widely adopted [9, 23], though it remains purely descriptive and characterises what a generator produces without indicating whether players enjoy it. An ERA characterisation of the PCG Arena generators is presented in Section 4.4 (Figure 4.6).

2.4.2 Agent-Based Testing

AI agents used as player proxies allow scalable, repeatable assessment of playability and difficulty.

Playability gates. A level is deemed playable if a strong A* agent can complete it. This binary check is the most common first stage in PCG evaluation pipelines [33, 29, 11].

Difficulty proxies. Agent-derived metrics include jump count (the number of jumps the A* agent performs, interpreted as difficulty), completion fraction (progress along the x-axis if the level is not fully completable), and virtual damage from stochastic "human-like" agents that model input timing inaccuracies [14].

Agent-based evaluation is fast and reproducible, but agent bias is a central concern: superhuman agents can solve levels humans cannot, and difficulty estimates depend on the specific agent policy. The mismatch also goes the other way: many levels that human players can complete are unsolvable by standard A* agents, which lack the ability to backtrack, use special power-ups, or exploit advanced movement techniques. These bi-directional limitations are rarely quantified in the literature.

2.4.3 Human Evaluation Studies

Human studies remain the gold standard for PCG evaluation but vary widely in scale and methodology.

Pairwise forced-choice. The Mario AI Championship [21] asked 15 judges to play two levels and select "which was more fun?" without further defining "fun." This protocol directly inspired PCG Arena's design.

Likert-scale ratings. Mariño et al. [12] used 7-point Likert scales (ordered response scales from "strongly disagree" to "strongly agree") for enjoyment, aesthetics, and difficulty with $n = 37$ participants. Summerville et al. [30] collected ratings across 85 design metrics and showed that inter-rater agreement imposes ceilings on achievable metric–human correlations. An automated metric cannot correlate with human ratings any better than humans correlate with each other, so reporting an inter-rater agreement baseline is important when interpreting metric–human correlations.

Large-scale web studies. Pedersen et al. [16] collected 240 game pairs via a web deployment of *Infinite Mario Bros* variants for pairwise affective-preference analysis. A follow-up by Shaker et al. [22] collected 600 game-pair comparisons via an uncontrolled web applet, though the number of unique participants was not reported. Both studies date from 2010–2011 and predate ML-based generators such as GANs, LSTMs, and diffusion models.

These studies show that while human empirical evaluation provides the most direct quality signal, it is expensive, typically small-scale ($n < 40$ in controlled settings), and conducted as isolated, one-time events. Earlier work on automatic personalised content generation [20] and design-pattern analysis [4] explored the relationship between level structure and player preferences, but likewise relied on small participant pools. Because the largest prior web studies were conducted before modern neural generators existed, contemporary Mario level generators have not yet been compared systematically in a shared human study.

2.4.4 Comparison of Approaches and Their Limitations

Table 2.2 summarises the evaluation methods employed across the most representative papers in the Mario PCG literature.

Paper	Year	PCG algorithm	S	A	ERA	H	ML
Smith & Whitehead	2010	Launchpad (rhythm-based)	✓		✓		
Pedersen et al.	2010	Notch (parameterised)	✓			✓	✓
Shaker et al. (Champ.)	2011	Comp. entries (mixed)	✓	✓		✓	
Shaker et al. (GE)	2012	Grammatical Evolution	✓		✓		
Horn et al.	2014	Multiple (comparative)	✓		✓		
Marino et al.	2015	GE + Notch (compared)	✓			✓	
Summerville & Mateas	2016	LSTM column generator	✓	✓			✓
Summerville et al.	2017	Multiple (8 generators)	✓	✓		✓	✓
Volz et al. (MarioGAN)	2018	DCGAN + CMA-ES	✓	✓			
Nam et al.	2024	Stochastic "human-like" agents	✓	✓		✓	
PCG Arena (ours)	**2026**	**14 generators (all paradigms)**	✓	✓	✓	✓	✓

Table 2.2: Evaluation methods used in Mario PCG research. S = structural metrics; A = agent-based; ERA = Expressive Range Analysis; H = human study; ML = learned predictor.

The works in Table 2.2 show how Mario PCG evaluation has developed from structural analyses, through agent-based playability checks and expressive-range characterisations, to small-scale human studies and learned predictors of player behaviour. Across this body of work, three recurring problems stand out:

Metrics–experience misalignment. Mariño et al. [12] showed that two generators rated identically by all computational metrics were rated *significantly differently* by human players on enjoyment. Summerville et al. [30] further demonstrated that inter-rater agreement imposes ceilings on achievable metric–human correlations, with aesthetics being particularly noisy. Automated metrics are useful characterisation tools, but they are unreliable proxies for perceived quality.

Small-n and one-time studies. Most human studies use 15–37 participants in controlled settings and were conducted as isolated events, often years apart. Moreover, when evaluations are not blind and players know which algorithm produced a level, prior expectations can bias their judgments.

Absence of a community-standard reward model. Search-based and RL-based generators require fitness or reward functions, yet no validated reward model for Mario level quality exists because human preference data across diverse generators has been unavailable.

PCG Arena addresses these gaps with a persistent, blind, web-based platform for human evaluation. The collected preferences can be used to (a) rank generators with statistical confidence, (b) test whether automated metrics correlate with human judgment, and (c) train preference-aligned generators.

2.5 Rating Systems for Pairwise Comparisons

PCG Arena uses pairwise comparisons as its primary data collection protocol. This section reviews the mathematical foundations underlying the platform's rating engine.

2.5.1 The Elo Rating System

Modern skill-rating systems build on the *Bradley-Terry model* [1]: each item i is assigned a strength parameter π_i, and the probability that item i is preferred to item j is

$$P(i \succ j) = \frac{\pi_i}{\pi_i + \pi_j}. \tag{2.1}$$

Elo [6] operationalised this model for competitive chess. Each player carries a scalar rating R; the expected score of player A against player B is

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}}. \tag{2.2}$$

After an observed outcome $S_A \in \{0, 0.5, 1\}$, the rating updates as $R'_A = R_A + K(S_A - E_A)$, where K is a fixed gain constant. Elo's simplicity made it the standard in chess, tennis, and early online games, but a constant K cannot distinguish a well-established rating from a newcomer's. Bayesian extensions address this limitation by tracking rating uncertainty explicitly.

2.5.2 The Glicko-2 Rating System

Glickman [7] recast Elo in a Bayesian framework (*Glicko*), modelling each player's rating as a Gaussian $\mathcal{N}(\mu, \sigma^2)$ where the *rating deviation* (RD) σ quantifies uncertainty. Between rating periods, RD grows to reflect increased uncertainty from inactivity.

Glicko-2 [8] extends Glicko with a third parameter, *volatility* σ_v, which captures how erratic an entity's performance is. The update pipeline converts ratings to an internal scale, estimates variance and improvement from outcomes, iteratively updates volatility, and then updates RD and rating. On the internal Glicko-2 scale the rating deviation is denoted $\phi = \mathrm{RD}/\lambda$, where $\lambda = 173.7178$ is the scale conversion factor:

$$\phi' = \frac{1}{\sqrt{1/\phi^{*2} + 1/v}}, \quad \text{where } \phi^* = \sqrt{\phi^2 + \sigma_v'^2}, \tag{2.3}$$

$$\mu' \leftarrow \mu + \phi'^2 \sum_j g(\phi_j)(s_j - E_j), \tag{2.4}$$

where $g(\phi) = 1/\sqrt{1 + 3\phi^2/\pi^2}$ down-weights predictions involving uncertain opponents, s_j is the observed outcome, and E_j the expected outcome.

PCG Arena uses Glicko-2 with generators and levels treated as "players": each human vote corresponds to a match outcome. RD provides a built-in measure of rating reliability, and volatility accommodates generators whose perceived quality shifts as new levels are added. Implementation details and parameter choices are described in Section 3.6.

2.5.3 Arena-Style Evaluation Platforms

The *arena paradigm*, in which anonymous pairwise votes from a crowd are used to rank competing systems, has also been adopted outside games. In *Chatbot Arena*, Chiang et al. [2] deployed a web platform where users chat simultaneously with two anonymous LLMs and vote for the better response. Over 240,000 votes from more than 90,000 users have been collected, and rankings are computed via Bradley-Terry MLE. An active sampling strategy concentrates votes on close model pairs, using information-gain ideas similar to PCG Arena's AGIS algorithm. Chatbot Arena is the closest structural analogue to PCG Arena outside the games domain.

A complementary direction is *LLM-as-a-Judge*: Zheng et al. [35] studied using GPT-4 as a surrogate judge for pairwise evaluation and reported over 80% agreement with human preferences. PCG Arena's *Judge Function* plays a similar role on a smaller scale as a heuristic filter for labelling additional level pairs (Section 4.6).

In the games domain, Sarkar and Cooper [18] applied Glicko-2 to the human computation game *Paradox*, treating puzzle tasks as rated entities alongside players; this is a direct precedent for PCG Arena's use of Glicko-2 to rate game content. More broadly, Snow et al. [26] showed that aggregating labels from multiple non-expert annotators can match expert-level quality, supporting the assumption that a sufficient number of non-expert pairwise votes yields a reliable ranking.

2.6 Preference Learning and Alignment

PCG Arena's preference data supports both generator ranking and active generator improvement via preference learning. This section reviews two main approaches: Reinforcement Learning from Human Feedback (RLHF) and its simplified successor, Direct Preference Optimisation (DPO).

2.6.1 Reinforcement Learning from Human Feedback

RLHF trains a policy to maximise a reward signal derived from human preferences. Christiano et al. [3] introduced the modern recipe: learn a reward model $\hat{r}$ from pairwise comparisons of behaviour, then optimise the policy against $\hat{r}$ with reinforcement learning. Ziegler et al. [36] adapted the scheme to language models, adding a KL

penalty $\beta\,\mathrm{KL}[\pi\|\pi_{\mathrm{ref}}]$ that keeps the fine-tuned policy close to the pretrained model. Stiennon et al. [27] scaled the approach to text summarisation and showed that a reward model learned from 64K human comparisons predicts preference better than ROUGE, the standard n-gram-overlap summarisation metric—direct evidence that learned preferences outperform hand-crafted proxies. Ouyang et al. [15] codified the three-step recipe (supervised fine-tuning, reward modelling, PPO) used to train InstructGPT and showed that alignment quality can matter more than model scale. The same principle applies to PCG: just as ROUGE fails to capture summary quality, automated level metrics fail to fully capture fun, aesthetics, or challenge appropriateness [12]; PCG Arena's paired-comparison voting is the game-design analogue of RLHF's preference collection phase.

2.6.2 Direct Preference Optimization

Rafailov et al. [17] observed that the optimal RLHF policy can be expressed in closed form as a function of the reward model. Substituting this solution back into the reward-model loss yields *Direct Preference Optimisation* (DPO), which fine-tunes the policy directly on preference pairs without training a separate reward model or running an RL loop:

$$\mathcal{L}_{\mathrm{DPO}}(\pi_\theta;\pi_{\mathrm{ref}}) = -\mathbb{E}_{(x,\,y_w,\,y_l)}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w|x)}{\pi_{\mathrm{ref}}(y_w|x)} - \beta\log\frac{\pi_\theta(y_l|x)}{\pi_{\mathrm{ref}}(y_l|x)}\right)\right] \tag{2.5}$$

where y_w and y_l are the preferred and dispreferred completions, β controls the deviation from the reference policy π_{ref}, and σ is the logistic sigmoid. The gradient increases the log-likelihood of preferred outputs and decreases that of dispreferred ones, weighted by how much the current policy already agrees. DPO matched or exceeded PPO-based RLHF on sentiment, summarisation, and dialogue tasks while being simpler to implement and more stable to train.

MarioDPO applies DPO to Mario level generation. Preference pairs are constructed from PCG Arena votes: the level receiving the human vote becomes y_w, the rejected level becomes y_l. The DPO objective fine-tunes a GPT-2-based level generator to produce levels more aligned with human preferences, closing the loop from human evaluation back to generation. The training procedure is detailed in Chapter 5.

Chapter 3

PCG Arena — System Design

This chapter presents the design and implementation of PCG Arena, a web-based platform for blind A/B testing of procedural content generators for Super Mario Bros. We describe the system architecture, game engine, battle and voting mechanics, the novel AGIS matchmaking algorithm, the Glicko-2 rating integration, the builder submission workflow, and the deployment infrastructure.

3.1 Requirements and Design Goals

The primary goal of PCG Arena is to enable rigorous, human-centered evaluation of PCG algorithms through pairwise comparisons. Translating this goal into a concrete system imposes several requirements:

1. **Blind comparison.** Players must not know which generator produced which level, so that prior expectations about algorithms do not bias their votes.
2. **Browser-based gameplay.** No installation should be required; the full Super Mario Bros experience must run in the browser with faithful physics.
3. **Uncertainty-aware ranking.** The rating system must quantify confidence in each generator's ranking.
4. **Efficient matchmaking.** With a limited player population, every battle should maximise information gain toward stable rankings.
5. **Open submission.** Any researcher should be able to submit their own generator and receive a rating with minimal friction.
6. **Rich telemetry.** Beyond the binary vote, the platform should capture gameplay trajectories, qualitative tags, and timing data for downstream analysis.

3.2 System Architecture

PCG Arena follows a three-tier web architecture with clear separation of concerns: a *presentation layer* (browser-based frontend), an *application layer* (RESTful backend), and a *data layer* (relational database). Table 3.1 summarises the technology choices for each layer.

Layer	Technology	Rationale
Frontend	React 18 + TypeScript 5.6 (Vite)	Canvas rendering; type safety
Backend	Python 3.12, FastAPI	Auto OpenAPI docs; Pydantic validation
Database	SQLite	Single-file; ACID; zero-config
Proxy	Caddy	Automatic TLS via Let's Encrypt
Container	Docker + docker-compose	Reproducible deployment

Table 3.1: PCG Arena technology stack.

3.2.1 Frontend

The frontend is a single-page application built with React 18 and TypeScript 5.6, bundled by Vite. Its source tree is organised into five top-level directories:

```
frontend/src/
  api/ # Backend communication (fetch wrappers)
  engine/ # Mario game engine (ported from Java)
  components/ # Reusable React UI components
  pages/ # Full-page route components
  contexts/ # React context providers (auth, theme)
```

Game rendering uses the HTML5 Canvas API for direct pixel manipulation, while the rest of the UI is standard React. Vite provides sub-second hot-module replacement during development and produces an optimised static bundle for production.

3.2.2 Backend

The backend is implemented in Python using FastAPI, chosen for its automatic OpenAPI documentation generation, native `async/await` support, and built-in request validation through Pydantic models. Key modules include:

- `main.py` — route definitions and middleware (CORS, rate limiting)
- `auth.py` — Google OAuth 2.0 and email/password authentication
- `glicko2.py` — Glicko-2 rating algorithm
- `matchmaking.py` — AGIS matchmaking algorithm
- `builders.py` — generator submission and management

- `stats.py` — analytics and data export endpoints

The API is versioned under the `/v1/` prefix and exposes endpoints for battles, votes, statistics, authentication, builder operations, and admin management. Administrative endpoints require either session-based admin access or an API key.

3.2.3 Database

SQLite was selected as the database engine due to its single-file storage (simplifying backups and deployment), ACID transaction guarantees, and zero-configuration operation. The schema is managed through 17 sequential SQL migrations and comprises the following tables:

Table	Purpose
`generators`	PCG algorithm metadata and ownership
`levels`	ASCII tilemap content with dimensions and content hash
`battles`	Pairwise comparison instances (left/right level pairing)
`votes`	User preference outcomes with telemetry and tags
`ratings`	Current Glicko-2 state (μ, RD, σ_v) per generator
`rating_events`	Audit log of all rating changes
`generator_pair_stats`	Confusion matrix data (pairwise win/loss counts)
`users`	Authenticated user accounts
`user_sessions`	Active login sessions
`email_verifications`	Email verification tokens
`password_resets`	Password reset tokens
`player_profiles`	Anonymous player identities
`player_sessions`	Player session tracking
`level_stats`	Aggregate per-level gameplay statistics
`play_trajectories`	Detailed player position data for heatmaps
`level_features`	Structural feature extraction (tiles, enemies, gaps)
`schema_migrations`	Migration bookkeeping

Table 3.2: Database schema: all 17 tables.

3.3 Game Engine

A critical component of PCG Arena is the browser-based Super Mario Bros engine, which was ported from the Java-based Mario AI Framework [10] to TypeScript. The port required faithful preservation of gameplay physics while adapting rendering and input handling to browser constraints.

3.3.1 TypeScript Port

The porting process involved four main translation steps. The port was carried out largely by hand, with assistance from AI-based coding tools for routine translation patterns. Ensuring faithful physics reproduction required iterative testing and manual corrections, particularly for collision detection, rendering synchronisation, and enemy-behaviour edge cases.

1. **Class structure.** Java classes were converted to TypeScript classes, preserving inheritance hierarchies (e.g. `MarioSprite` → `Mario`, `Enemy`).
2. **Physics constants.** All numerical constants were kept identical to the Java originals (Table 3.3).
3. **Rendering.** Java Swing/AWT rendering was replaced with the HTML5 Canvas API (`CanvasRenderingContext2D`).
4. **Input.** Keyboard event handling was reimplemented using browser `KeyboardEvent` listeners.

The key engine modules are:

```
engine/
  MarioGame.ts # Game loop (requestAnimationFrame)
  MarioWorld.ts # World state, collision, game logic
  MarioLevel.ts # ASCII tilemap parser
  MarioSprite.ts # Base sprite class (physics, rendering)
  sprites/ # Mario, enemies, items, projectiles
  graphics/ # Camera, sprite sheets
  effects/ # Particle effects
```

Parameter	Value
Ground inertia	0.89
Air inertia	0.89
Gravity	3.0
Jump velocity	−1.9
Walk speed	0.6
Run speed	1.2

Table 3.3: Core physics constants preserved from the Java Mario AI Framework.

The game loop runs at 30 ticks per second, matching the original Java framework. Rendering runs separately at 60 FPS via `requestAnimationFrame`, interpolating between physics frames. This decoupling preserves the exact physics behaviour of the original framework (which assumes 30-tick updates) while providing visually

smooth animation in the browser. Physics updates are fully deterministic, ensuring consistent gameplay across sessions.

3.3.2 Level Format

Levels are stored as plain-text ASCII tilemaps, where each character encodes a tile type or entity spawn point. The format supports 37 distinct tile characters, summarised in Table 3.4.

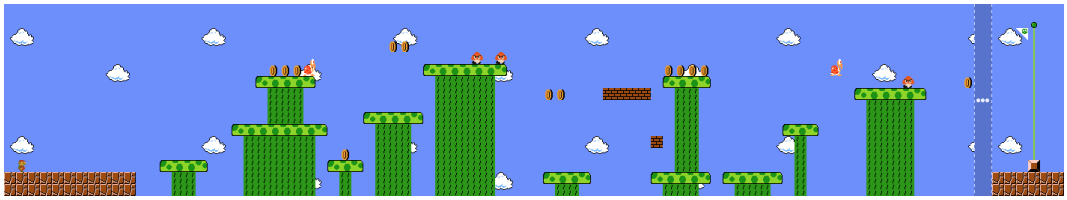
Char	Meaning	Char	Meaning
`-`	Air (empty)	`Q`	Question block (coin)
`X`	Solid floor	`!`	Question block (coin, alt)
`S`	Brick block	`C`	Coin block
`#`	Pyramid block	`U`	Mushroom block
`D`	Used block	`L`	1-Up block
`%`	Jump-through platform	`1`	Invisible 1-Up block
`|`	Platform background	`2`	Invisible coin block
`?`	Question block (mushroom)	`o`	Free-standing coin
`@`	Question block (mushroom, alt)	`M`	Mario spawn
`E`	Goomba	`F`	Flag (level exit)
`g`	Goomba (alt)	`t`	Pipe (empty)
`G`	Winged Goomba	`T`	Pipe (flower)
`k`	Green Koopa	`<`	Pipe top left
`K`	Winged Green Koopa	`>`	Pipe top right
`r`	Red Koopa	`[`	Pipe body left
`R`	Winged Red Koopa	`]`	Pipe body right
`y`	Spiky	`*`	Bullet Bill body
`Y`	Winged Spiky	`B`	Bullet Bill head
		`b`	Bullet Bill neck

Table 3.4: Complete ASCII tile legend (37 characters).

A typical level file consists of 16 rows (the default height) and up to several hundred columns, stored with newline-separated rows. For example, the original Super Mario Bros World 1-1 is encoded as a 202×16 character grid. Figure 3.1 shows a segment of this level.

```
------------------------------------------------------------------------------...------
------------------------------------------------------------------------------...------
------------------------------------------------------------------------------...------
------------------------------oo--------------------------------------------...------
---------------------------------------g-g------------------------------------...------
---------------------ooor---------%%%%%%%-------------oooo----------R-----------...------
--------------------%%%%%----------|||||--------------%%%%----------------g----o...------
----------------------|||-----------|||||----oo---SSSS--||-------------%%%%%%----...------
----------------------|||-----------|||||----------------||--------------||||-----...------
----------------------|||-----%%%%%-|||||----------------||--------------||||-----...------
-------------------%%%%%%%%----|||--|||||----------------||-------%%%----||||-----...------
--------------------||||||-----|||--|||||-------------U-||--------|-----||||-----...------
--------------------||||||--o--|||--|||||----------------||--------|-----||||-----...---F--
-M----------%%%%---||||||-%%%-|||--|||||----------------||--------|-----||||-----...---#--
XXXXXXXXXXX---||----||||||--|--|||--|||||----%%%%-----%%%%%-%%%%%-|-----||||-----...XXXXXX
XXXXXXXXXXX---||----||||||--|--|||--|||||-----||-------|||---|||--|-----||||-----...XXXXXX
```

(a) ASCII tilemap.



(b) Rendered view of the same segment.

Figure 3.1: Original Super Mario Bros Level (World 1-1 segment). `M` marks Mario's spawn; `X` is solid ground; `U` and `S` are question blocks; `g` is a Goomba; `F` denotes level exit; `-` is empty air.

3.3.3 Level Validation

Every submitted level undergoes format validation before insertion into the database. The validation procedure, implemented in the backend's `seed.py` module, checks:

1. **Height bounds:** the number of rows must satisfy $10 \leq h \leq 20$.
2. **Width bounds:** the number of columns must satisfy $150 \leq w \leq 250$. In practice, all submitted generators produce levels with widths between 150 and 208 columns; the lower bound of 150 corresponds to the standard screen width of the original Super Mario Bros.
3. **Rectangular shape:** all rows must have identical length.
4. **Character whitelist:** every character must belong to the set of 37 allowed tiles (Table 3.4).

Levels that fail any check are rejected with a descriptive error message. The platform performs *format-level* validation only, without automated playability or

reachability analysis (e.g. checking that the exit is reachable from the spawn). Content hashes are computed for deduplication.

3.4 Battle and Voting System

The core user experience is the *battle*: a blind A/B test in which the player compares two levels produced by different generators, not knowing which generator produced which level.

3.4.1 Battle Flow

A battle proceeds through five stages:

1. **Battle request.** The frontend calls `POST /v1/battles:next`.
2. **Matchmaking.** The AGIS algorithm (Section 3.5) selects two generators; one random level is drawn from each.
3. **Gameplay.** The player plays both levels sequentially (labelled "Left" and "Right"). During play, the engine records a trajectory of Mario's position and death events.
4. **Voting.** After completing both levels, the player chooses one of four options: *Left is Better*, *Right is Better*, *Tie*, or *Skip*. The "Skip" option, used when the player feels unable to judge or encounters a technical issue, does not affect ratings.
5. **Rating update.** The backend updates Glicko-2 ratings for both generators atomically within a database transaction (Section 3.6).

Figure 3.2 shows the main gameplay interface. After completing both levels, the voting panel (Figure 3.3) allows the player to cast their preference and optionally tag each level.



Figure 3.2: Main gameplay view: the player experiences a procedurally generated level in the browser-based Mario engine.



Figure 3.3: Voting panel shown after both levels are played, with preference buttons. Tag selection is accessible by hovering over the level (see Figure 3.4).



Figure 3.4: Tag selection popup shown when the player hovers over a level. Players may assign up to three descriptive labels per level.

3.4.2 Tagging System

In addition to the preference vote, players may optionally tag each level in the battle with up to three descriptive labels drawn from a fixed vocabulary of seven tags (Table 3.5), chosen to cover enjoyment, engagement, creativity, difficulty, playability, and visual correctness.

Tag	Description
`fun`	The level was enjoyable to play
`boring`	The level lacked engagement
`creative`	The level showed novel or interesting design
`too_hard`	The level was excessively difficult
`too_easy`	The level was trivially easy
`impossible`	The level appeared to be uncompletable
`broken_graphics`	The level had visual or rendering issues

Table 3.5: The seven available tags for level characterisation.

Tags are stored as boolean columns in the `votes` table and aggregated in `level_stats` for downstream analysis (Section 4.5).

3.5 Matchmaking: AGIS Algorithm

3.5.1 Problem Definition

Given n generators with current Glicko-2 ratings and uncertainties, the matchmaking problem is to select a pair for the next battle that maximises information gain toward accurate final rankings. Naive strategies are suboptimal:

- **Uniform random** wastes battles on already-converged pairs.
- **Most-uncertain-first** neglects head-to-head comparisons between similarly rated generators.
- **Round-robin** ignores rating information and pairs dissimilar generators.

To address these shortcomings, we designed *AGIS* (Adaptive Glicko-Informed Selection), a two-stage matchmaking algorithm that combines rating uncertainty, skill similarity, and pairwise coverage into generator selection. Its components adapt standard ideas from information-theoretic experimental design (uncertainty sampling and coverage balancing) into a lightweight, Glicko-2-aware matchmaker for PCG evaluation.

3.5.2 Stage 1: First Generator Selection

Each generator i receives a selection weight w_i composing an uncertainty term and a games-played term:

$$w_i = \left(1 + \widetilde{\mathrm{RD}}_i\right)^2 \cdot \mathrm{boost}(\mathrm{games}_i) \tag{3.1}$$

where $\widetilde{\mathrm{RD}}_i = (\mathrm{RD}_i - \mathrm{RD}_{\min})/(\mathrm{RD}_{\max} - \mathrm{RD}_{\min})$ is the normalised rating deviation. The *boost* function provides a 4× weight multiplier for generators with zero games, linearly decaying to 1× at `MIN_GAMES` (default: 30). After convergence, a mild quality bias is applied:

$$\mathrm{boost}_{\mathrm{converged}} = 0.8 + q_{\mathrm{bias}} \cdot \min\Big(1,\ \max\big(0,\ \tfrac{\mu_i - 600}{800}\big)\Big) \tag{3.2}$$

where $q_{\mathrm{bias}} = 0.2$ is the configurable quality bias strength. The weights are normalised to sum to 1, forming a categorical (discrete) probability distribution; the first generator is then drawn from this distribution via weighted random sampling.

3.5.3 Stage 2: Second Generator Selection

The intuition behind Stage 2 is that the most informative battle is one between generators of *similar* strength (maximising discriminative power) whose ratings are still *uncertain* (maximising information gain per battle), while ensuring that every pair accumulates enough head-to-head data for a reliable confusion matrix. The four weighted components below formalise these intuitions.

Given the first generator g_1, the second is selected using a composite pair weight:

$$w_{1,j} = \alpha \cdot S(g_1, g_j) \; + \; \beta \cdot U(g_j) \; + \; \gamma \cdot I(g_1, g_j) \; + \; \mathrm{cov}(g_1, g_j) \tag{3.3}$$

The four components are:

- **Rating similarity** $S(g_1, g_j) = \exp\!\left(-\Delta\mu^2 / 2\sigma_s^2\right)$, a Gaussian kernel with $\sigma_s = 150$ penalising large rating gaps.
- **Uncertainty** $U(g_j) = 1 + \widetilde{\mathrm{RD}}_j$, favouring uncertain generators (range $[1, 2]$).
- **Information gain** $I(g_1, g_j)$, incorporating the joint RD and a match-quality estimate.
- **Coverage bonus** $\mathrm{cov}(g_1, g_j)$: for pairs with fewer than `TARGET_BATTLES_PER_PAIR` (default: 10) battles, an exponential bonus $2 \cdot e^{-c/3}$ is applied, where c is the current battle count; otherwise a residual 0.1 is used.

3.5.4 Parameters

Table 3.6 lists the configurable AGIS parameters and their defaults, which are set via environment variables.

Parameter	Description	Default
α	Rating similarity weight	0.5
β	Uncertainty weight	0.3
γ	Information gain weight	0.2
`MIN_GAMES`	Games before "converged"	30
`TARGET_BATTLES_PER_PAIR`	Min. battles per pair	10
`RATING_SIGMA`	Similarity kernel σ_s	150.0
`QUALITY_BIAS`	Quality bias strength q_{bias}	0.2

Table 3.6: AGIS algorithm parameters and default values.

3.6 Glicko-2 Rating System

PCG Arena uses the Glicko-2 rating system [8] to maintain uncertainty-aware rankings. Each generator carries three parameters:

- **Rating (μ):** the skill estimate, displayed on a scale centred at 1000.
- **Rating Deviation (RD, ϕ):** the uncertainty in the rating. High RD indicates an unreliable estimate; low RD indicates a well-established rating.
- **Volatility (σ):** the expected fluctuation in performance over time.

The mathematical foundations of the Glicko-2 system (the Bradley-Terry model, expected score computation, and the update equations for ϕ' and μ') are presented in Section 2.5.2. Here we describe the implementation-specific aspects of how PCG Arena applies these formulas.

3.6.1 Rating Update Procedure

After each battle, the winning generator receives a score of 1, the loser 0, and ties yield 0.5 for both. Using the expected score and update equations from Section 2.5.2, the implementation proceeds as follows:

1. Convert ratings to the internal Glicko-2 scale (centred at 0, divided by $\lambda = 173.7178$). This constant equals $400/\ln 10 \approx 173.7178$ and bridges the Elo convention (400 points per tenfold win-probability change) with the natural logistic function used internally by Glicko-2.
2. Compute the variance v from the opponent's RD.
3. Compute $\Delta = v \cdot g(\phi_B)(s - E)$, where s is the actual score.
4. Update volatility σ' via the iterative Illinois algorithm (system constant $\tau = 0.5$).
5. Update RD and rating using Equations 2.3–2.4.
6. Convert back to the display scale and clamp to $[100, 3000]$.

All rating updates are performed atomically within a single database transaction, with an audit record written to the `rating_events` table.

3.6.2 Configuration

Table 3.7 lists the Glicko-2 constants used by PCG Arena.

Parameter	**Value**
Initial rating (μ_0)	1000
Initial RD (ϕ_0)	350
Initial volatility (σ_0)	0.06
System constant (τ)	0.5
Scale factor (λ)	173.7178
Minimum RD	30
Maximum RD	350
Minimum rating	100
Maximum rating	3000

Table 3.7: Glicko-2 configuration parameters.

The RD of each generator provides a natural 95% confidence interval for its true rating: $\mathrm{CI}_{95\%} = [\mu - 1.96\,\mathrm{RD},\ \mu + 1.96\,\mathrm{RD}]$. However, the public leaderboard displays only the point-estimate rating; confidence intervals are not shown on the platform's frontend. Figure 3.5 shows the leaderboard as seen by players.



Figure 3.5: Public leaderboard ranking generators by their Glicko-2 rating.

Clicking a generator reveals detailed statistics including rating history, per-opponent win rates, a level gallery, and tag distributions (Figure 3.6).

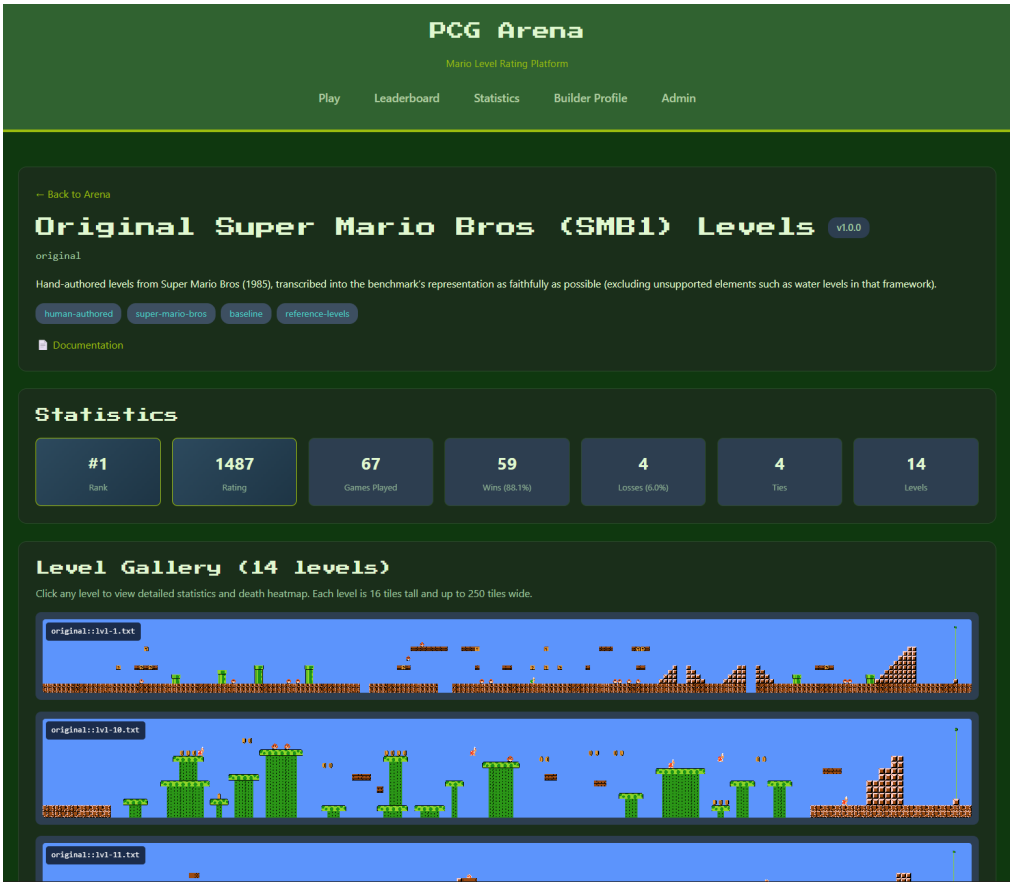


Figure 3.6: Generator detail page showing level gallery and performance statistics.

For researchers, the platform also exposes a per-level analytics view (Figure 3.7) that overlays the level map with recorded player traces, death locations, and tag counts.

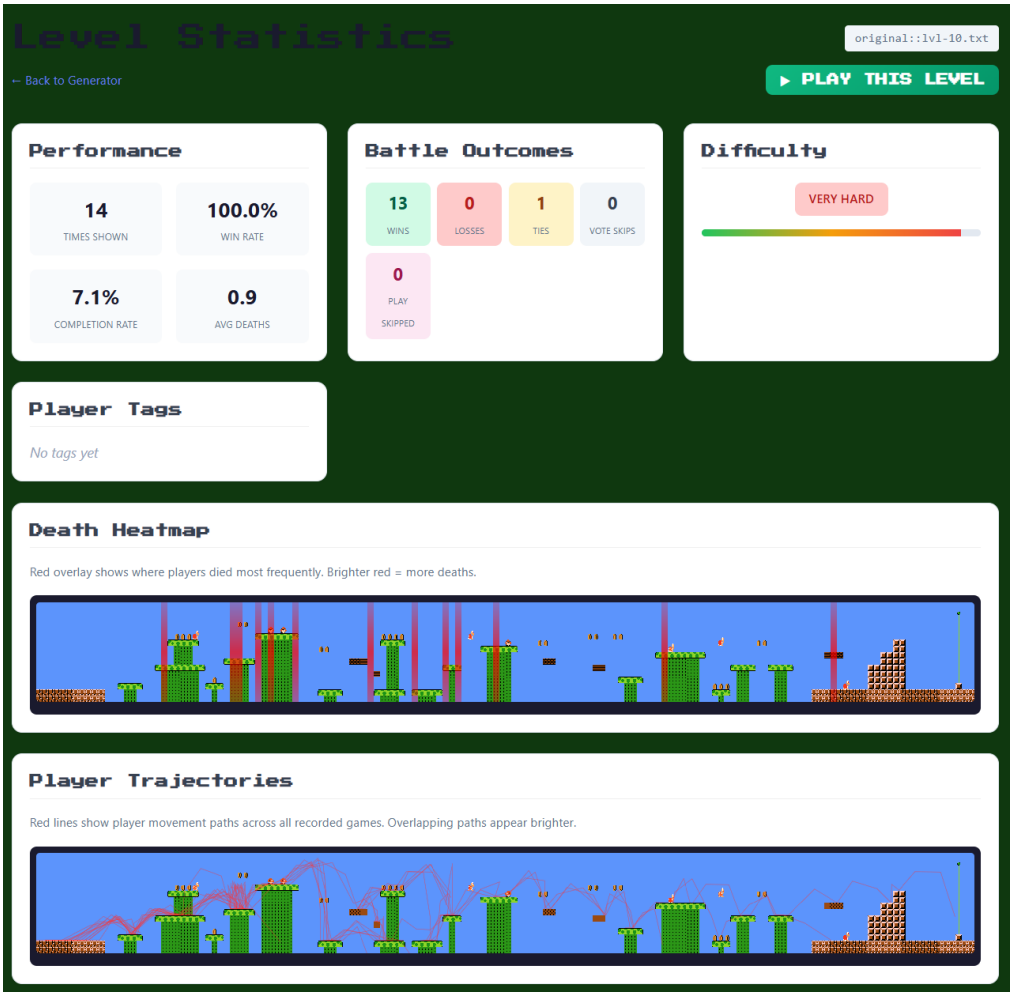


Figure 3.7: Detailed Level Statistics page from PCG Arena. The interface supports qualitative inspection of individual levels through trajectory overlays, death locations, tag counts, and level visualization.

3.7 Builder Profile and Generator Submission

Authenticated users may submit their own generators through the Builder Profile page. The submission workflow is:

1. The user provides generator metadata: name, version, description, and a URL to the generator's source repository.
2. The user uploads a ZIP archive containing level files in the ASCII tilemap format.
3. The backend validates every level (Section 3.3.3). Levels that fail validation are rejected with detailed error messages.
4. Valid levels and the generator record are inserted into the database.

5. An initial Glicko-2 rating of 1000 ± 350 is assigned.
6. The generator appears on the leaderboard immediately and enters the matchmaking pool.

Each user may own at most three generators. When a user wishes to remove a generator, the system distinguishes two cases. If the generator has already participated in battles, it is *soft-deleted*: marked as inactive and excluded from future matchmaking, but its records (levels, ratings, votes) are retained so that historical analyses and the confusion matrix remain consistent. If the generator has never appeared in a battle, it is *hard-deleted* (fully removed from the database) since no other data depends on it.

Figure 3.8 shows the Builder Profile page.

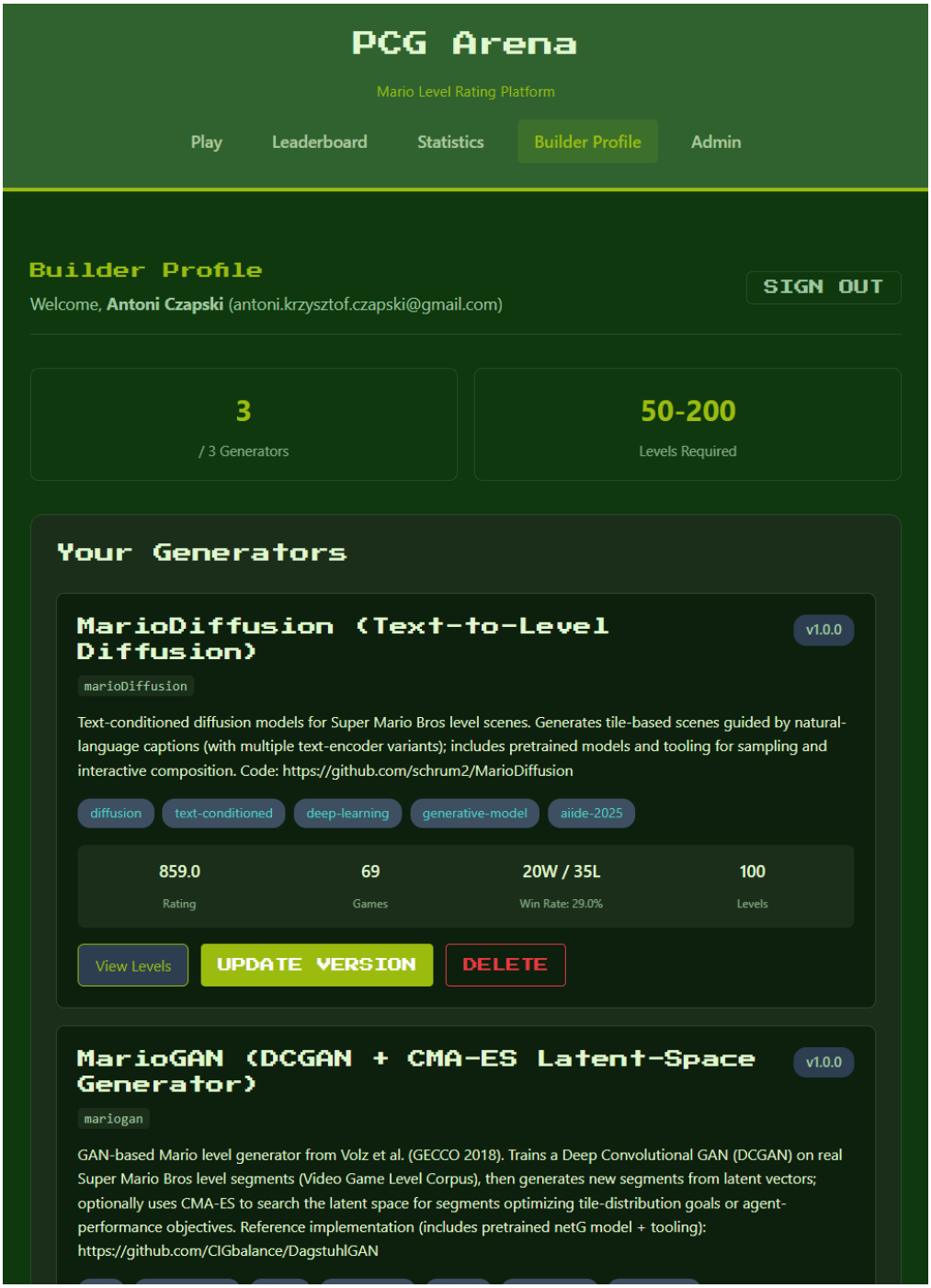


Figure 3.8: Builder Profile page for submitting and managing generators.

3.8 Authentication

PCG Arena supports two authentication methods:

- **Google OAuth 2.0.** The frontend opens a Google sign-in popup; the returned ID token (JWT) is sent to `POST /v1/auth/google`, where the backend verifies

it against Google's servers. Google users are automatically marked as email-verified.

- **Email and password.** Users register with an email, password, and display name. Passwords are hashed with bcrypt (work factor 12). A verification email is sent via Twilio SendGrid containing a unique token (64 hex characters, 24-hour expiry).

Upon successful authentication, the backend issues a session cookie (`arena_session`) containing a 256-bit cryptographically random token. Sessions expire after seven days. HTTP-only cookies prevent client-side script access.

Administrative access is granted to users whose email addresses appear in the `ARENA_ADMIN_EMAILS` configuration list.

3.9 Deployment and Infrastructure

PCG Arena is deployed on Google Cloud Platform using a single Compute Engine instance (`e2-micro`, Ubuntu 22.04 LTS). The backend runs inside a Docker container orchestrated by `docker-compose`, with the SQLite database file bind-mounted from the host for persistence across container rebuilds.

Caddy acts as the reverse proxy and handles:

- Automatic TLS certificate provisioning and renewal (Let's Encrypt).
- HTTP-to-HTTPS redirection.
- Reverse proxying API requests to `localhost:8080`.
- Static file serving for the frontend production build.

The platform is accessible at `https://pcg-arena.com`. The backend binds to `127.0.0.1:8080` and is not exposed to the public internet directly; all external traffic passes through Caddy. Firewall rules allow only ports 80 and 443. Database backups are performed via a scheduled script that copies the SQLite file to a timestamped archive.

Chapter 4

User Study — Measuring Player Preference

This chapter analyses the PCG Arena data as a user study of player preference. Section 4.1 describes the study protocol, the dataset, and the data-collection timeline. Next, to address RQ1, Section 4.2 derives a preference ranking of generators from blind pairwise votes, and Section 4.3 examines whether the anonymous player population is consistent enough for this aggregation to be meaningful. To address RQ2, Sections 4.4–4.5 test whether static level structure, gameplay trajectories, and descriptive tags can explain or replace the ranking, and Section 4.6 constructs a heuristic judge function that Chapter 5 uses to label additional training pairs.

The analysis is exploratory and relies on patterns observed in the data and its visualisations as the sample size is too small to support strong statistical claims.

4.1 Study Protocol and Dataset

Participants were recruited through university social-media channels, word of mouth among game-development and computer-science communities, and a small promoted Reddit campaign targeting game-development and procedural-generation communities.[1] Participation was anonymous: the platform assigned browser-based player and session identifiers persisted through cookies or local browser storage.

The promoted Reddit campaign recorded 62,407 impressions. Summing the country-level ad exports from 1 January–31 May 2026 gives a reported reach of 55,001 and 415 clicks (Figure 4.1), reflecting where the recruitment advertisement was shown and clicked.

[1] Recruitment post: `https://www.reddit.com/r/proceduralgeneration/s/pmlt0kw37q`.

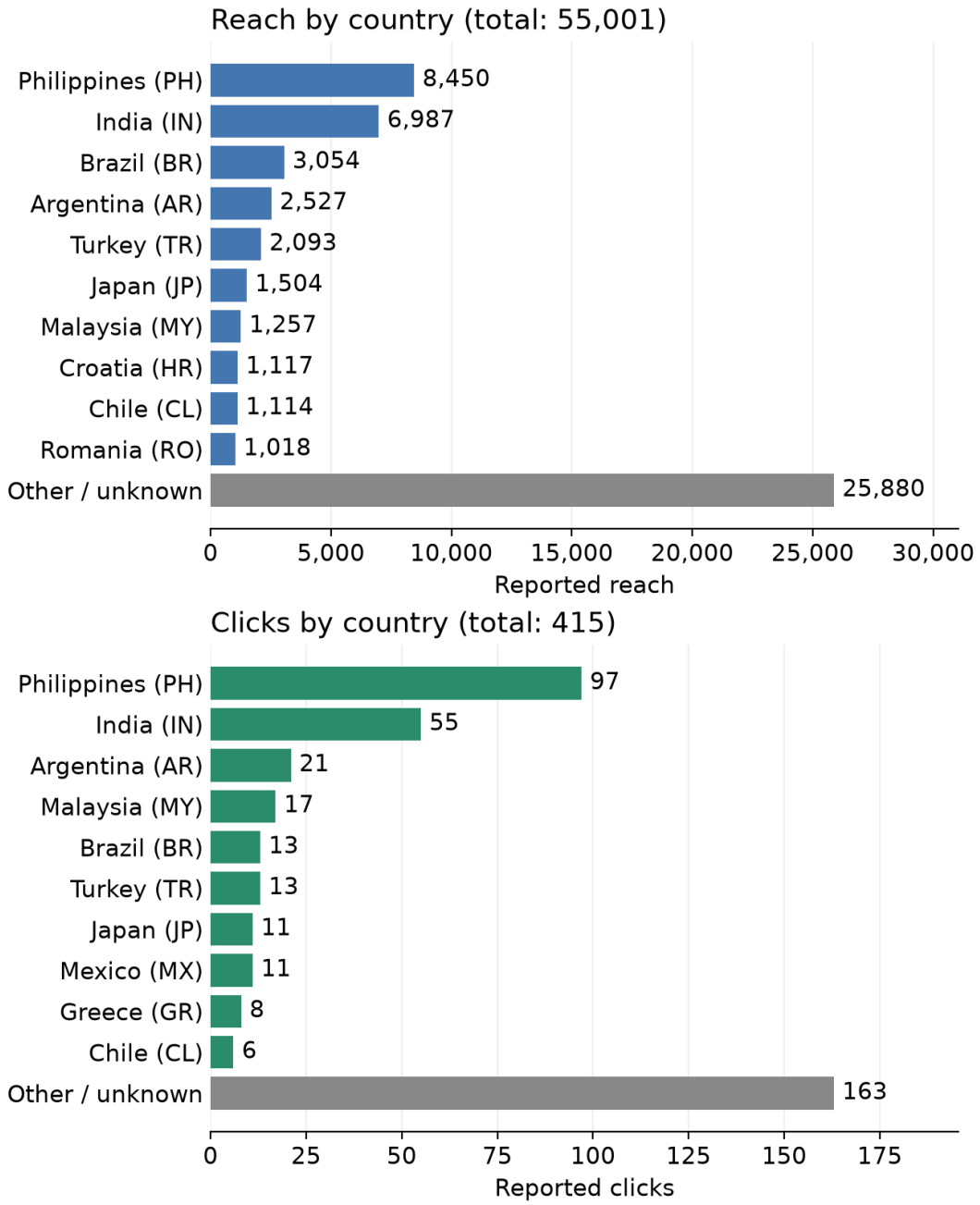


Figure 4.1: Reddit recruitment campaign: reported reach (top) and click count (bottom) by country from the country-level ad exports for 1 January–31 May 2026. Each panel shows the ten highest-count countries; the remaining countries and records without a country code are grouped as "Other / unknown" (total reach: 55,001; total clicks: 415).

Each battle used a blind A/B protocol. A player saw two levels from different generators, played one attempt on each side, and then selected the preferred side, a tie, or skip. Optional tags could be attached to either side. Restricting each side to a single attempt was a deliberate design choice: a multi-attempt protocol was also implemented and tested, but single-attempt battles yield substantially more pairwise

comparisons per unit of player time.

The May 2026 analysis export contains the data summarized in Table 4.1. Skip votes are included in engagement summaries but excluded from preference rankings. Ties are treated as 0.5 points for each side.

Table 4.1: Dataset summary for the exploratory analysis.

Quantity	**Count**
Vote rows analyzed	1,000
Anonymous player IDs in vote table	77
Level-stat records exported	1,104
Generator IDs	14
Per-side telemetry records	2,000
Played side records	1,968
Embedded trajectory records	1,968
Tag assignments	329

Vote outcomes are balanced between the two screen sides (418 left wins and 398 right wins, with 165 ties and 19 skips), and the assignment of levels to the left or right side was randomised. The median play duration per side is 8.0 seconds, with a small number of inactive-tab outliers.

The May 2026 export is the final snapshot of the data-collection process summarised in Table 4.2. MarioDPO (Chapter 5) was trained once on the January 2026 training snapshot and deployed without further changes. The analyses in this chapter use the May export because its larger sample size yields tighter confidence intervals while evaluating the same frozen model and baseline generators.

Phase	**Period**	**Data**	**Role in this study**
Baseline collection	launch → Jan. 2026	votes over the 13 baseline generators	—
Training snapshot	Jan. 2026	473 usable non-tie votes	judge calibration and DPO training pairs
MarioDPO deployment	Jan. 2026	—	model frozen
Full export	May 2026	1,000 votes, 14 generators	definitive analysis and generator evaluation

Table 4.2: Timeline of PCG Arena data snapshots and their roles. MarioDPO was trained once on the January 2026 training snapshot and deployed unchanged; the May export therefore evaluates the same frozen model.

4.2 Generator Preference Ranking

The primary goal of PCG Arena is to measure human preference between generators. For each non-skip battle, the selected side receives score 1, the rejected side receives score 0, and ties give 0.5 to both sides. Figure 4.2 shows the resulting generator leaderboard with 95% bootstrap confidence intervals over votes.

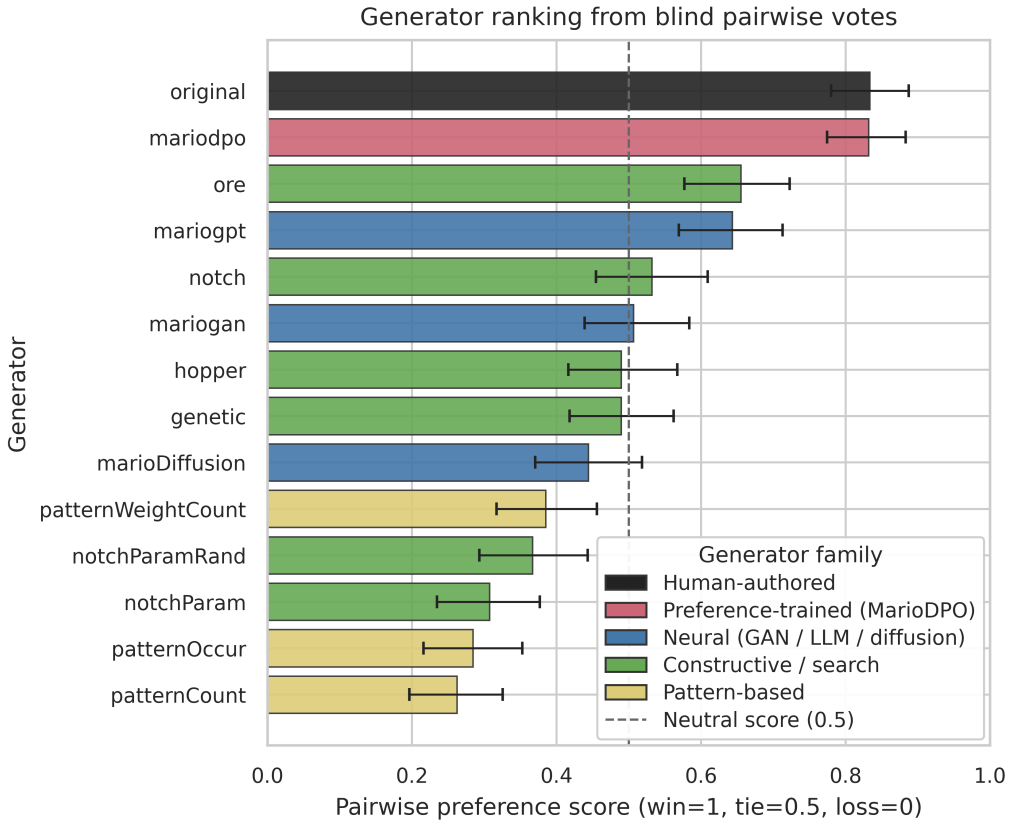


Figure 4.2: Generator ranking from blind pairwise votes. Bars show mean preference score, where win = 1, tie = 0.5, and loss = 0; horizontal intervals are 95% bootstrap confidence intervals obtained by resampling votes. Bar colour encodes the generator family (human-authored, preference-trained, neural, constructive/search, and pattern-based), as shown in the in-plot legend.

At the top of the ranking, the human-authored `original` baseline and `mariodpo` are nearly tied: `original` scores 0.833, while `mariodpo` scores 0.832. Both are well above the neutral 0.5 line. The next tier consists of `ore` (0.655) and `mariogpt` (0.643), followed by mid-table constructive and neural generators. The pattern-based generators occupy the bottom of the ranking: `patternWeightCount` scores 0.385, `patternOccur` scores 0.284, and `patternCount` scores 0.262.

As a first sanity check, the human-authored baseline remains at the top of the leaderboard, as expected if the votes reflect level quality. The export also includes MarioDPO, the preference-aligned generator developed in Chapter 5; we discuss its

results in Section 5.5.

Figure 4.3 gives the corresponding head-to-head matrix, with rows and columns ordered by the overall ranking. In this matrix, `original` and `mariodpo` achieve high win rates across most pairings, while the pattern-based generators lose consistently against nearly all opponents.

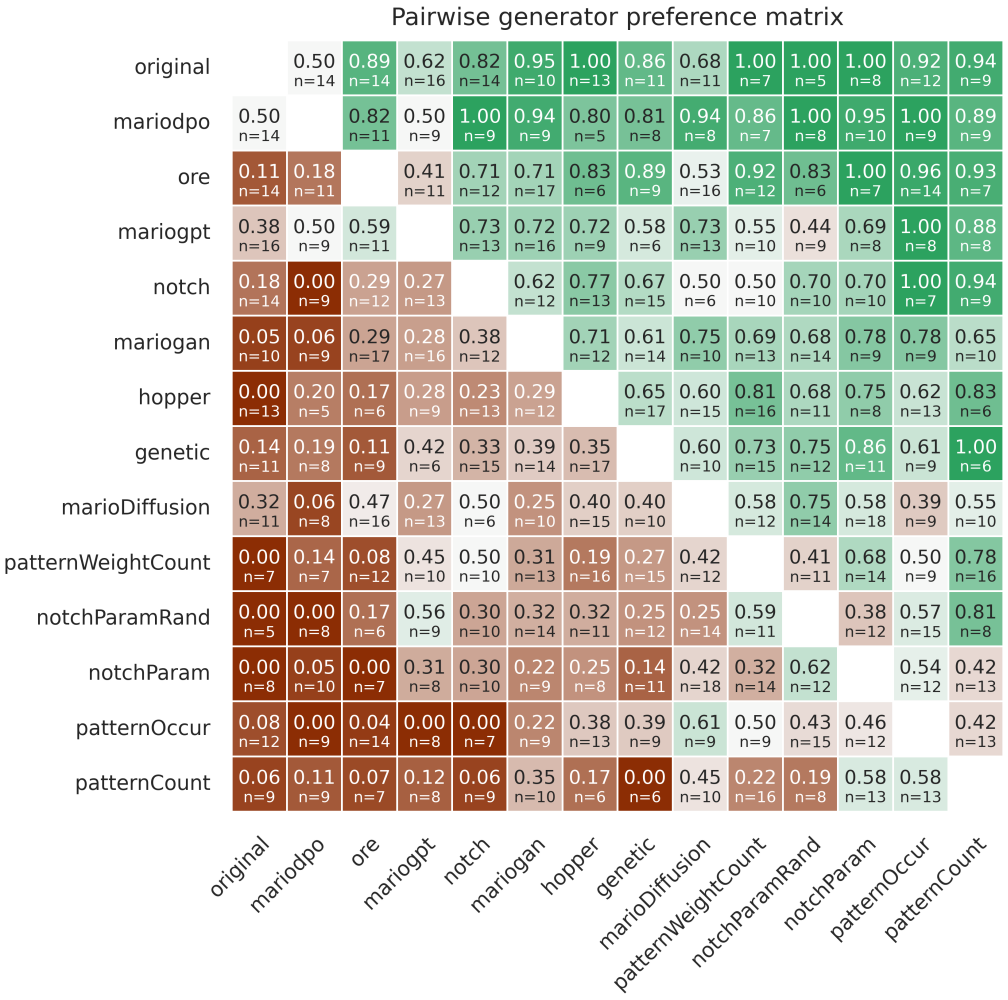


Figure 4.3: Pairwise generator preference matrix ordered by overall ranking. Each cell is the score rate for the row generator against the column generator, with ties counted as 0.5. Green indicates a higher score rate for the row generator.

4.3 Player Engagement and Preference Consistency

Aggregating individual votes into the single ranking of Section 4.2 requires that the underlying player population be reasonably consistent. This section examines how votes are distributed across anonymous player IDs and whether their preferences align. Figure 4.4 shows votes over time, votes per anonymous player ID, and cumulative vote contribution.

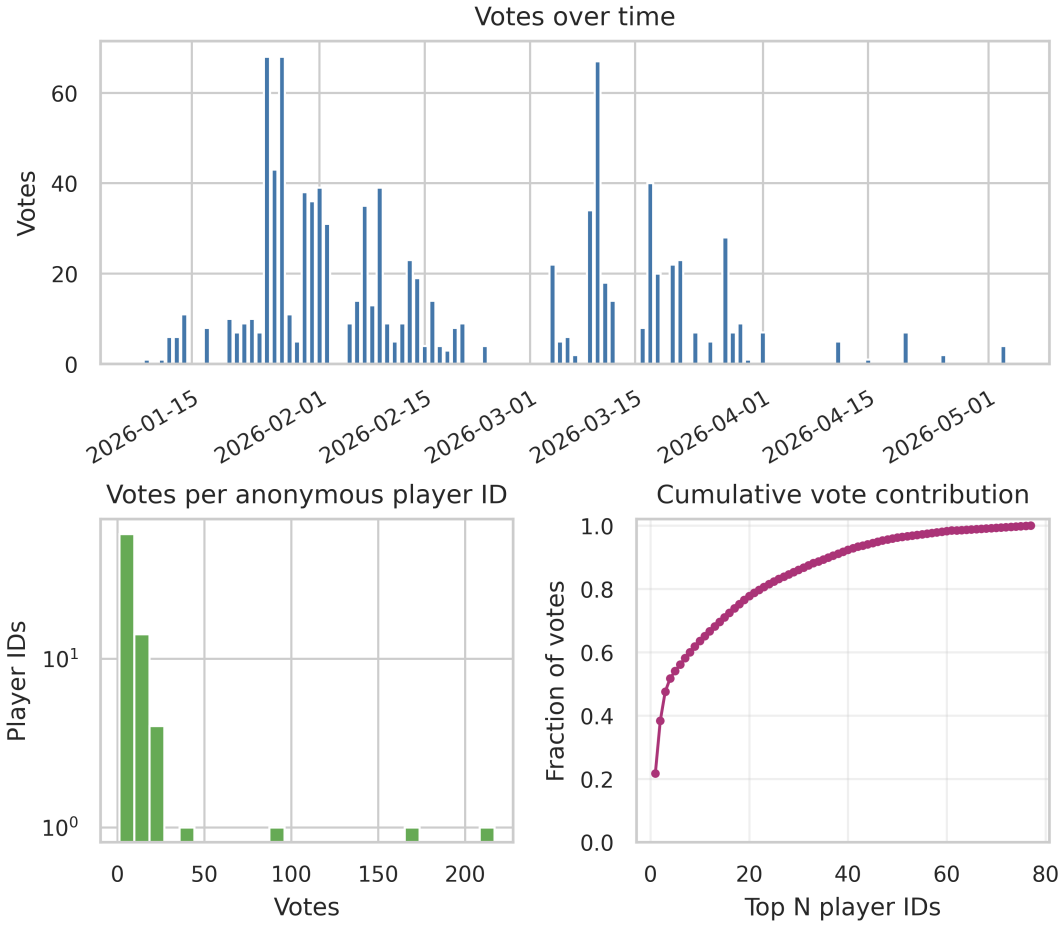


Figure 4.4: Engagement concentration across anonymous player IDs.

The vote distribution is heavily concentrated. Among the 77 anonymous player IDs in the vote table, the median player ID contributes 6 votes, while the most active ID contributes 217 votes. The top 5 IDs account for 54% of the analyzed votes, and the top 10 account for 63.5%. Consequently, aggregate generator scores reflect both broad participation and the preferences of a small number of highly active contributors.

Figure 4.5 examines the player-by-generator preference matrix for anonymous player IDs with at least five votes. Each cell is the mean score assigned by a player ID when that generator appeared. Missing player-generator combinations are filled with the neutral value 0.5 only for visualization and clustering.

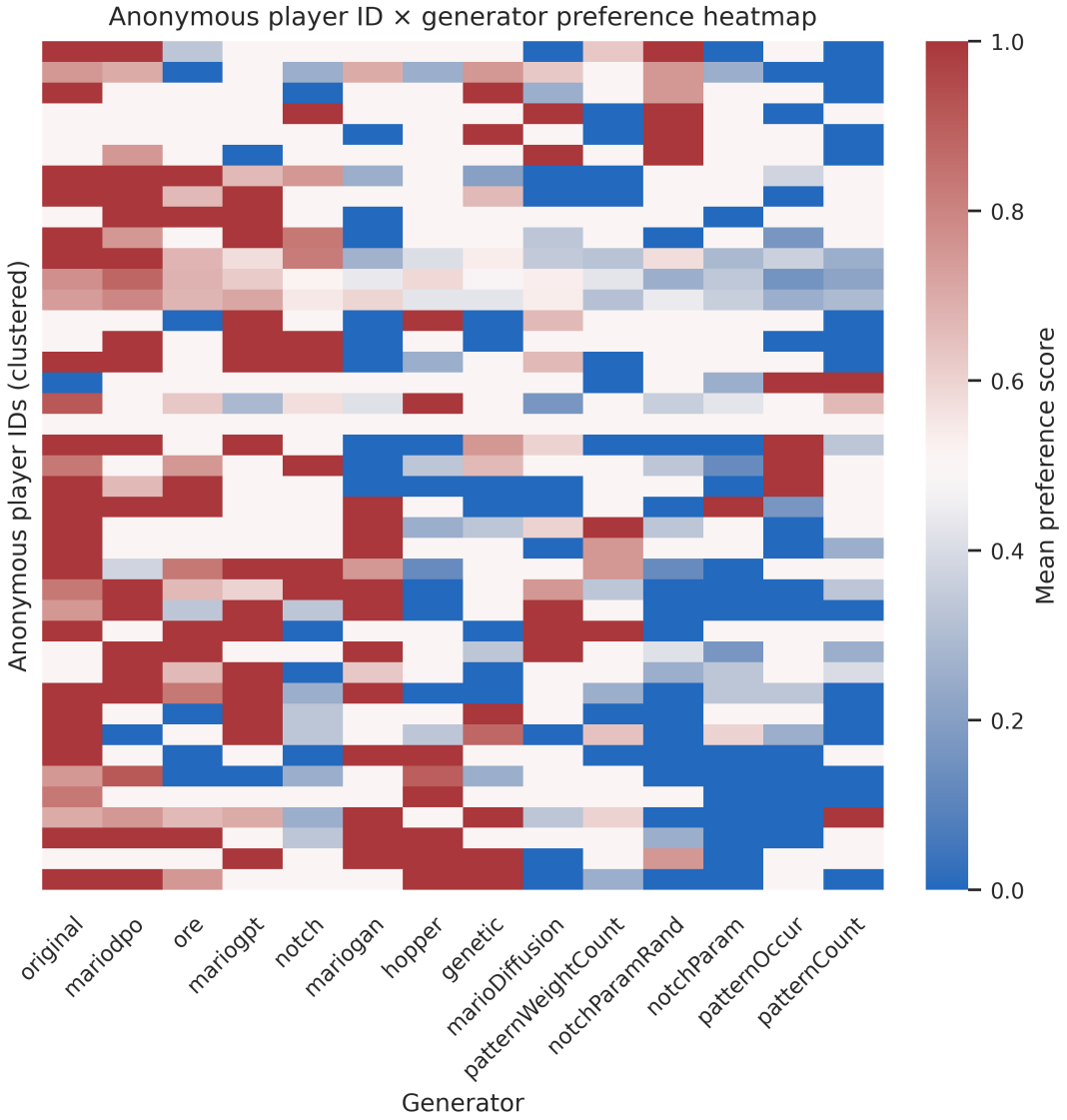


Figure 4.5: Preference heatmap of anonymous player IDs against generators. Each row is a single anonymous player ID with at least five votes, and each cell is that player's mean preference score for the generator in that column (win = 1, tie = 0.5, loss = 0). Rows are ordered by hierarchical clustering (Ward's method on Euclidean distances between players' generator-score vectors) so that players with similar preference profiles are placed next to one another. Columns follow the global generator ranking.

Despite some individual variation, players largely agree on which generators perform well and which perform poorly: most rows score the top-ranked generators highly and the bottom-ranked generators low, with only mild disagreement on the mid-table generators. Because the votes reflect a shared preference signal across players, they can be combined into a single leaderboard (Section 4.2) and used as a training signal in Chapter 5.

4.4 What Static Metrics Can and Cannot Tell Us

This section examines what can be learned about a generator from the levels themselves, without player telemetry. Each ASCII tilemap is summarised by a set of descriptive quantities: how flat or hilly its terrain is, how many enemies it contains, how often gaps occur, how many rewards it offers, and how similar its columns are to one another. Computing the same quantities for every generator allows us to test whether any of these structural properties correlate with the player preference scores from Section 4.2.

We compute the metrics directly from the ASCII level files. Linearity is the coefficient of determination (R^2) of a least-squares line fitted to the playable surface profile; it measures how well a straight line explains the surface, so a higher R^2 corresponds to flatter, more predictable terrain, while a lower value indicates a more irregular profile. Density metrics count solid, enemy, reward, and gap structure per tile or per level column. Leniency combines gap density, enemy density, maximum gap width, rewards, and blocks into a single descriptive score; because leniency definitions in the PCG literature are not mutually comparable, we use this formulation only for within-study comparisons among the 14 generators. Diversity is estimated with tile/column entropy, the unique-column ratio, the compression ratio, and gzip normalized compression distance (NCD).

Table 4.3 reports these metrics per generator, ordered by preference score.

Generator	Score	Levels	Linearity	Leniency	Gap dens.	Enemy dens.	NCD
original	0.83	15	0.02	0.61	0.18	0.08	0.83
mariodpo (ours)	0.83	200	0.06	0.67	0.14	0.09	0.81
ore	0.65	100	0.11	0.63	0.16	0.06	0.85
mariogpt	0.64	100	0.10	0.55	0.22	0.06	0.76
notch	0.53	100	0.12	0.70	0.10	0.02	0.75
mariogan	0.51	100	0.04	0.62	0.10	0.07	0.73
hopper	0.49	100	0.17	0.63	0.11	0.12	0.73
genetic	0.49	100	0.03	0.73	0.05	0.03	0.77
marioDiffusion	0.44	100	0.05	0.44	0.28	0.02	0.78
patternWeightCount	0.38	100	0.07	0.72	0.08	0.13	0.87
notchParamRand	0.37	100	0.14	0.74	0.07	0.02	0.73
notchParam	0.31	100	0.05	0.71	0.07	0.02	0.70
patternOccur	0.28	100	0.05	0.70	0.08	0.17	0.87
patternCount	0.26	100	0.15	0.72	0.05	0.25	0.83

Table 4.3: Static expressive metrics by generator (May 2026 export), ordered by the preference score of Section 4.2. Linearity is the surface-fit R^2; NCD is the mean within-generator normalised compression distance.

As Table 4.3 shows, static structure acts only as a coarse filter. The four highest-scoring generators have similar static profiles (linearity between 0.02 and 0.11, le-

niency between 0.55 and 0.67, gap density between 0.14 and 0.22, and enemy density between 0.06 and 0.09), yet their preference scores span 0.64 to 0.83; none of the static columns explains why players prefer `original` and `mariodpo` over `ore` and `mariogpt` by nearly 0.2. No individual metric separates strong from weak generators across the table either: the highest leniency belongs to `notchParamRand` (0.74, score 0.37), the highest structural diversity by NCD to `patternWeightCount` and `patternOccur` (0.87, scores 0.38 and 0.28), and the lowest-ranked `patternCount` has above-average linearity. Extreme values do flag degenerate designs: `patternCount`'s enemy density (0.25) is roughly triple that of any top-tier generator, and `marioDiffusion`'s leniency (0.44) is the lowest in the corpus, but such extremes only identify obvious failures.

The answer to RQ2 is therefore negative: structural metrics cannot replace human votes when ranking generators, consistent with the gap between computational metrics and player experience reported by Mariño et al. [12]. Figure 4.6 illustrates this in the linearity–leniency plane: each generator occupies a distinct region, so the metrics describe structural differences between generators, yet neither the position nor the spread of these regions tracks the preference ranking.

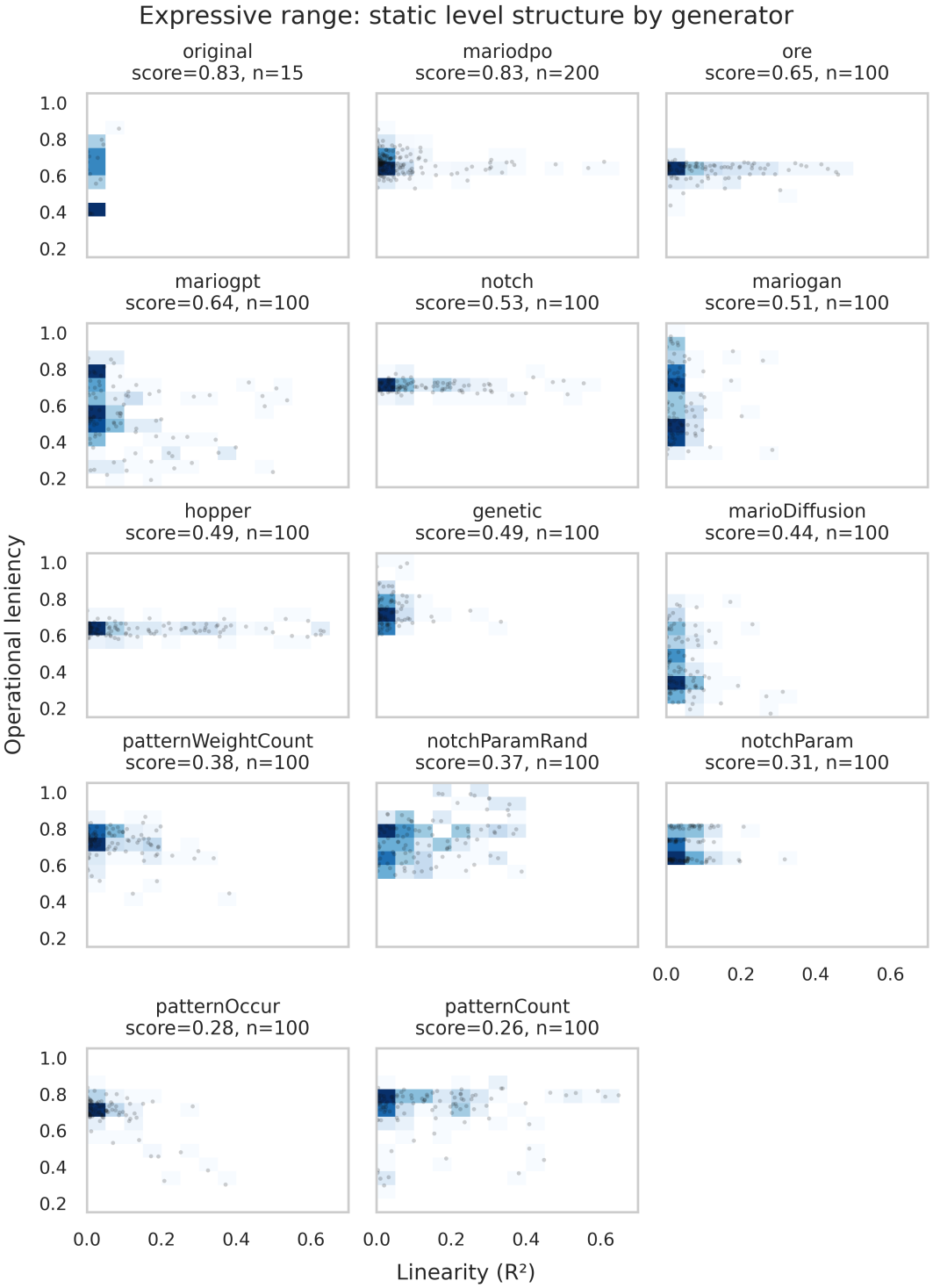


Figure 4.6: Expressive-range view of static level structure. Each subplot shows levels from one generator in the linearity–leniency plane, ordered by preference ranking.

4.5 Why Levels Lose: Trajectories and Tags

Although static structure does not explain the ranking, the battle records include two additional data sources that help explain why levels *lose*: where players fail (gameplay trajectories) and how they label the levels (tags).

For each generator, Figure 4.7 overlays all recorded Mario paths as thin red traces, showing where players actually moved during their single attempt per level.

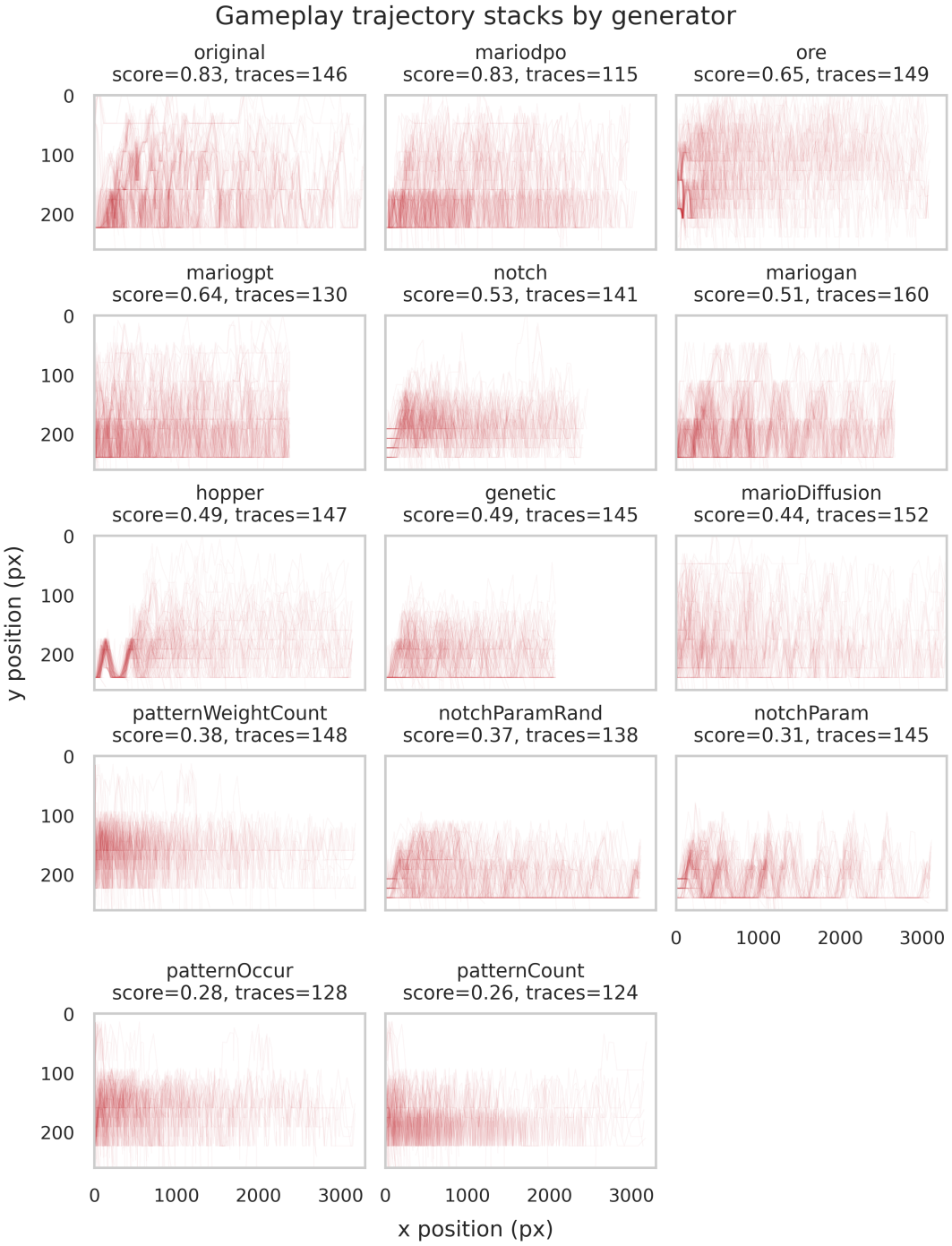


Figure 4.7: Stacked gameplay trajectories by generator. Each subplot overlays all recorded trajectories for one generator as thin red traces on a white background, ordered by preference score. The vertical axis is inverted to match screen coordinates.

The trajectory plots show clear differences in player progress. For the top generators, traces extend far into the level and span a wide vertical band: the median maximum progress is 1126.5 pixels for `original` and 988.0 pixels for `mariodpo`. For the lowest-ranked generators, traces cluster near the start and terminate early: `patternCount` has a median maximum progress of only 174.5 pixels, and 63% of

its recorded deaths fall into a single early horizontal bin. In these levels, players encounter an immediate obstacle within the first screens.

Player-assigned tags add a qualitative diagnostic layer to the binary votes. Figure 4.8 summarizes tag frequency, tag rate by generator, tag co-occurrence, and outcome distribution when each tag is present. The most common tags are `impossible` (87 assignments), `fun` (56), `too_hard` (55), `boring` (49), `creative` (39), `broken_graphics` (22), and `too_easy` (21).

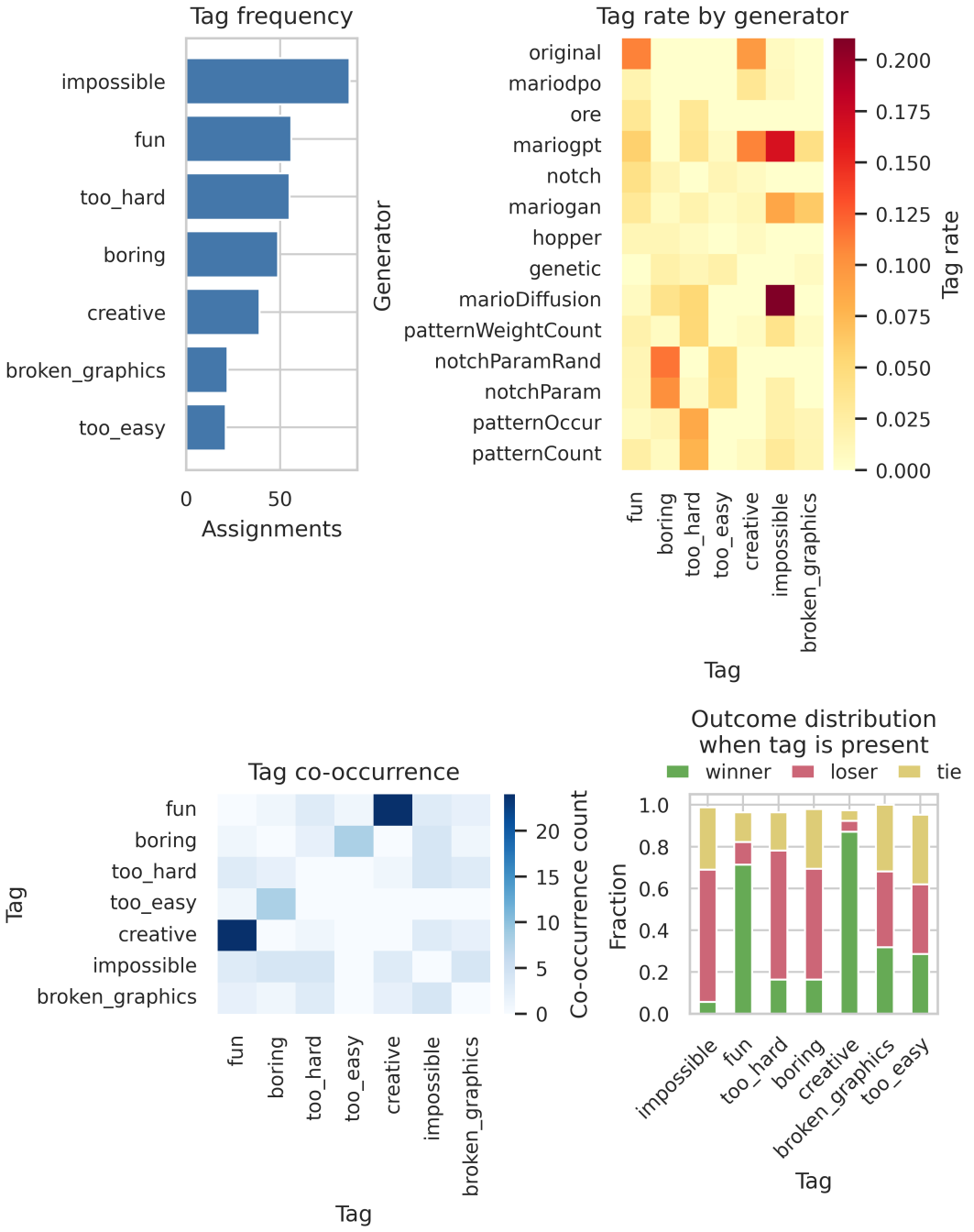


Figure 4.8: Tag semantics summary. The panels show tag frequency, tag rate by generator, tag co-occurrence, and the outcome distribution for tagged sides.

Positive tags correspond closely with winning outcomes: sides tagged `creative` win 87% of the time and sides tagged `fun` win 71% of the time. Negative tags behave differently, `impossible`, `too_hard`, and `boring` are much more common on losing sides. The tag-by-generator heatmap also separates generator failure modes, distinguishing generators marked as impossible or too hard from those rejected as boring or visually broken.

Together, trajectories and tags point to the same mechanism: the main diagnosable cause of lost battles is early frustration. Levels that kill the player repeatedly near the start are the same levels tagged `impossible` and `too_hard`, and they consistently lose. While `fun` and `creative` co-occur with winning levels, Section 4.4 showed no simple structural formula for positive enjoyment. This asymmetry motivates the design of the judge function in the next section.

4.6 The Judge Function

Chapter 5 trains a generator on preference pairs, and the 473 usable human votes in the January training snapshot (Table 4.2) are too few on their own to fine-tune a sequence model. We therefore need an automatic way to label additional level pairs. Because no combination of static metrics reproduces the human ranking (Section 4.4), we do not attempt to model enjoyment directly; instead, we target the failure modes identified in Section 4.5. The judge function J_{final} is designed as a coarse filter that separates flawed levels from plausible ones.

The judge is a weighted sum of five interpretable terms:

$$\begin{aligned} J_{final} = w_{style}\frac{1}{1+D_M} - w_{gap}\,\mathit{GapPenalty} + w_{comp}\,\mathit{Completion} \\ + w_{vert}\,\frac{\sigma_y}{50} + w_{flow}\,(1-\mathit{Hesitation}), \end{aligned} \tag{4.1}$$

where D_M is the Mahalanobis distance from a level's feature vector to the centroid of the original Nintendo levels, rewarding proximity to the human-authored region of feature space; *GapPenalty* penalises fragmented, pit-heavy ground; *Completion* is the fraction of the level traversed; σ_y is the standard deviation of Mario's vertical position along a trajectory; and *Hesitation* is the fraction of trajectory time spent at near-zero horizontal velocity.

Because the last three quantities depend on gameplay trajectories and the judge must score newly generated levels that no human has played, they are computed by running an A* agent through each candidate level. Using an agent as a stand-in for a player is standard practice in PCG evaluation (Section 2.4.2), though the agent's superhuman play policy reinforces the role of J_{final} as a coarse filter.

The weights (Table 4.4) were selected by a grid search maximising pairwise agreement with the human votes under fixed term signs. The gap penalty receives

the largest weight, consistent with early frustration being the clearest failure mode in the arena data (Section 4.5), while the remaining terms incorporate the structural and trajectory features from Sections 4.4 and 4.5.

Term	Weight	Role
w_{gap}	0.50	penalise the dominant failure mode: fragmented, pit-heavy structure
w_{style}	0.32	reward proximity to the original-level centroid
w_{vert}	0.26	reward vertical variety
w_{comp}	0.20	reward completion of the level
w_{flow}	0.10	penalise hesitation and poor flow

Table 4.4: Judge-function weights, obtained by grid search over interpretable weights maximising pairwise agreement with human votes.

Aggregated by generator on the January snapshot, J_{final} correlates with human win rate at $r_s = 0.736$ ($p = 0.0027$). Because the weights were fitted on the same snapshot, this correlation represents an in-sample check across the 14 generators: it confirms that the judge reproduces the broad quality tiers by placing degenerate, frustrating generators below plausible ones.

This level of discrimination is sufficient for how Chapter 5 uses J_{final}: the judge only labels pairs of levels with a clear score margin so that training includes diverse examples of flawed structure to avoid, while the strongly upweighted human votes provide the positive preference signal. The practical test of this setup is whether the resulting generator wins blind human votes after deployment.

4.7 Summary: A Reliable but Sparse Signal

The chapter yields five main findings:

1. **Measurement works**: blind pairwise votes produce a stable ranking, with `original` and `mariodpo` nearly tied at the top (≈0.83) and pattern-based generators at the bottom.
2. **Players agree**: the same generators score high or low across the anonymous population, so the aggregate ranking reflects shared preference.
3. **Static metrics cannot rank**: the top generators have near-identical static profiles and no metric column separates strong from weak; structure acts only as a coarse filter for degenerate designs.
4. **Failure is diagnosable**: early, concentrated deaths and `impossible`/`too_hard` tags mark losing levels, while `fun` and `creative` align with wins but show no structural recipe.

5. **A coarse judge suffices for labelling**: combining the failure signals yields J_{final}, which reproduces the quality tiering in-sample ($r_s = 0.736$) by catching easy-to-spot failures.

Two properties of the vote signal matter for what follows. It is reliable: anonymous players largely agree on which generators are good (Section 4.3). It is also sparse: the January training snapshot contains only 473 usable non-tie votes (Table 4.2), far too few to fine-tune a sequence model directly. The next chapter therefore trains on the human votes as the primary preference signal and uses the judge only to broaden negative coverage—closing the loop from measurement to generation.

Chapter 5

MarioDPO — Preference-Aligned Level Generation

5.1 Motivation: From Sparse Votes to a Training Signal

The previous chapter established three results. First, blind pairwise votes produce a reliable ranking signal, as anonymous players largely agree on which generators perform well (Section 4.3). Second, no automated metric substitutes for that signal (Section 4.4), which motivates training MarioDPO directly on player preferences. Third, frustrating and uncompletable structure is reliably identifiable, and the judge function of Section 4.6 can label it cheaply and at scale. However, the January 2026 training snapshot contains only 473 usable non-tie votes, which is orders of magnitude smaller than the preference datasets used to align language models (e.g., the 64K comparisons of Stiennon et al. [27]) and too small to fine-tune a level generator on human votes alone.

MarioDPO therefore combines three ingredients. The base model is the published MarioGPT checkpoint, so the generator already produces valid Mario structure before preference optimisation begins. The 473 human pairs, upweighted 10×, anchor training to human judgements and carry the positive preference signal. Finally, the judge function labels 2,606 additional synthetic pairs to steer the model away from common failure modes without diluting the human preference objective.

MarioDPO consists of four components:

1. A column-major level representation that preserves the vertical structure of Mario tilemaps while remaining compatible with autoregressive sequence modelling;
2. A base generator reusing the MarioGPT checkpoint;
3. A preference dataset combining the human PCG Arena votes with synthetic pairs labelled by the judge function developed in Section 4.6; and

4. A DPO fine-tuning stage that aligns the base generator with the resulting preference signal.

The design separates *validity* from *preference*. Validity is enforced through representation choices, start/end constraints, and deterministic post-processing (Section 5.4), whereas preference is learned from human votes and the judge function. Because a level can be playable yet unengaging, MarioDPO trains the model to generate levels that players prefer in blind A/B tests while handling structural validity separately.

Figure 5.1 summarises the composition of the training data. The MarioGPT checkpoint provides the structural prior, PCG Arena provides human comparisons, the judge function expands these comparisons into a larger preference dataset, and DPO aligns the generator with the resulting preference signal.

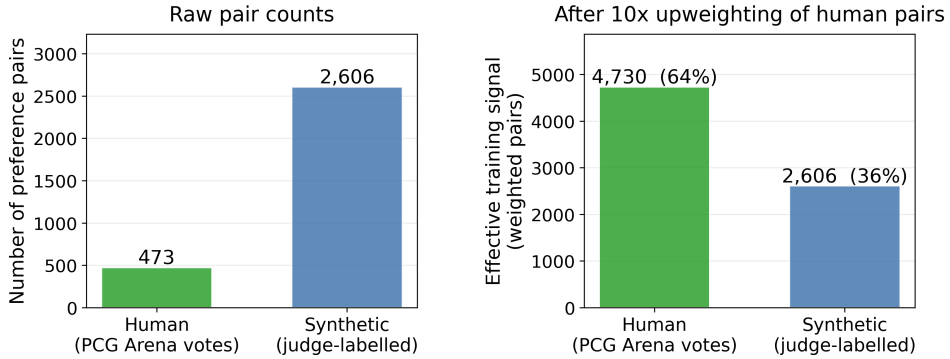


Figure 5.1: MarioDPO preference dataset: raw pair counts (left) and the same counts after applying the 10× upweighting of human PCG Arena pairs (right). The few hundred human comparisons contribute roughly two thirds of the effective training signal, while pairs labelled by the judge function (Section 4.6) primarily broaden coverage.

5.2 Level Representation and Base Model

5.2.1 Column-Major Tokenisation

Mario levels in this thesis are represented as ASCII tilemaps: each level is a matrix of characters, with rows corresponding to vertical tile positions and columns corresponding to horizontal progress through the level. The standard height is 16 tiles; widths vary between generators, with the corpus average equal to 193.8 tiles. The vocabulary contains 37 distinct tile characters, dominated by empty space (`-`), solid blocks (`X`), pipes (`|`, `<`, `>`, `[`, `]`), platforms (`%`), question blocks (`?`, `Q`), coins (`o`), and enemies such as Goombas (`g`) and Koopas (`k`).

A naive row-major serialisation would place horizontally adjacent tiles next to

one another in the sequence, but would separate vertically adjacent tiles by an entire row length. This is undesirable for Mario levels, where local vertical relationships are essential: enemies stand on platforms, pipes occupy multiple rows, and jumpable structures depend on the configuration of several tiles in a column. MarioDPO therefore uses a column-major representation. Conceptually, the level is decomposed into a sequence of 16-character columns,

$$c_t = (x_{1,t}, x_{2,t}, \ldots, x_{16,t}), \tag{5.1}$$

where $x_{r,t}$ is the tile at row r and column t. The autoregressive model then predicts the next column in a flattened column-major sequence. This preserves vertical locality while still allowing standard left-to-right generation.

Table 5.1 summarises the representation choices. Post-processing acts on the same column units when enforcing ground continuity, start safety, and flag placement.

Table 5.1: MarioDPO level representation and tokenisation choices.

Property	**Choice**
Level format	ASCII tilemap
Height	16 rows
Average width in corpus	193.8 tiles
Vocabulary size	37 tile characters
Sequence order	Column-major, left to right
Validity constraints	Safe start, bounded gaps, end flag, post-processing

Training examples are obtained by sliding a window over the column-major sequence. At inference time, generated segments are decoded back into 16-row tilemaps and then passed through the validity filters described in Section 5.4.

5.2.2 Base Model

MarioDPO reuses the published MarioGPT checkpoint [28] as its reference policy π_{ref}. MarioGPT is an autoregressive decoder already fine-tuned on column-major Mario levels, so it produces syntactically valid SMB structure (ground in the lower rows, consistent multi-tile pipes, blocks above reachable space, and enemies on supporting tiles) before any preference optimisation begins. Starting from this checkpoint allows the DPO stage to focus on shifting the policy towards preferred levels.

5.3 Preference Dataset Construction

The DPO dataset contains two sources of pairwise supervision: direct human votes from PCG Arena and synthetic pairs scored by the judge function developed in

Section 4.6 (Figure 5.1).

Human pairs. Each non-tie PCG Arena vote is converted into a pair (y_w, y_l), where y_w is the level selected by the player and y_l is the rejected level. Votes marked as ties or skips are excluded because they do not define a strict preference.

Synthetic pairs. To expand the dataset, candidate levels are scored with J_{final} (Section 4.6, Equation 4.1). For a pair of levels (a, b), the higher-scoring level becomes the winner if the score margin exceeds 0.1. This margin avoids training on ambiguous pairs where the judge itself is nearly indifferent; across the selected synthetic pairs, the mean score difference is 0.441.

5.4 DPO Fine-Tuning and Inference-Time Filtering

The DPO objective is the one introduced in Equation 2.5. In the MarioDPO setting, the prompt x is a level prefix, while y_w and y_l are the preferred and rejected level continuations. The reference policy π_{ref} is the MarioGPT checkpoint, and the trainable policy π_θ is initialised from it.

The training configuration used in the experiments is shown in Table 5.2. The value $\beta = 0.1$ controls how strongly DPO penalises divergence from the reference policy. A small learning rate is used because the dataset is preference-rich but small relative to typical language model corpora.

Table 5.2: MarioDPO training configuration. Both stages run in fp16 on a single 6 GB GPU.

Stage / parameter	**Value**
Base model	MarioGPT checkpoint (DistilGPT-2)
Vocabulary	37 ASCII tile characters
Context length	1024 tokens
Batch size	32
DPO β	0.1
Learning rate	10^{-5}
Epochs	3
Human-pair weight	10×
Synthetic-pair weight	1×

Training increases the likelihood ratio of preferred levels relative to rejected levels, while the reference-policy term prevents catastrophic drift away from valid Mario syntax. Intuitively, if the SFT model assigns similar probability to two level

continuations but humans or the judge prefer one, DPO shifts probability mass toward the preferred continuation. If the current model already strongly prefers the winner, the gradient is small.

Inference-time filtering. Preference alignment alone does not guarantee that every sampled level is valid. MarioDPO therefore uses rejection sampling at inference time. For each requested level the model generates a batch of $N = 10$ candidates. Each candidate is normalised and passed through a deterministic post-processing pass that ensures Mario has a valid spawn point in the first columns, that a flag exists in the last columns, and that both regions contain solid ground. The candidates are then scored with J_{final}, and the highest-scoring candidate is selected. This post-processing step is separate from the learned model: the model learns preference, while the filter enforces hard validity constraints.

5.5 Results

MarioDPO is evaluated in the May 2026 PCG Arena export analysed in Chapter 4, under the same blind pairwise-preference protocol as all other generators.

In the May 2026 PCG Arena export, MarioDPO is the highest-ranked procedural generator. It obtains a preference score of 0.832, nearly tied with the original Nintendo levels at 0.833 and ahead of ORE (0.655) and MarioGPT (0.643). The pairwise matrix in Chapter 4 (Figure 4.3) further shows that MarioDPO achieves strong win rates across direct matchups against all other generators.



Figure 5.2: MarioDPO compared with other procedural generators: it is the strongest PCG method in the arena and the closest competitor to the original human-authored levels.

MarioDPO also improves substantially over MarioGPT (0.832 vs. 0.643), the checkpoint from which it was initialised. Because both models share the same architecture and pretraining corpus, this gain comes directly from preference alignment on the arena data.

5.6 Discussion

Three design choices account for MarioDPO's performance.

First, the base model starts from a strong structural prior. Reusing the Mar-

ioGPT checkpoint and a column-major representation gives the generator a bias toward valid, readable Mario geometry before preference optimisation begins. This avoids a common failure mode of pure search or random generation, where the optimiser spends most of its capacity rediscovering basic syntax.

Second, the judge function (Section 4.6) supplies broad negative coverage by flagging gap-fragmented and uncompletable levels, so DPO sees many clear negative examples while the human votes provide the positive preference signal.

Third, the preference dataset preserves the primacy of human judgement. Synthetic pairs make the dataset large enough for DPO, but the 10× weight on human votes ensures that the model is ultimately anchored to PCG Arena preferences. This design is important because J_{final} is only an approximation: it reproduces the coarse quality tiering, but it cannot capture nostalgia, aesthetic surprise, or all forms of level readability.

Together, these components close the loop that the platform was built to enable. PCG Arena supplies human comparisons; the judge function turns them into a scalable learning signal; and DPO converts that signal into a generator whose levels reach human-authored quality. The exported levels are packaged in the platform's seed format, ready for redeployment to gather the fresh votes that would drive the next iteration of the same loop.

Chapter 6

Conclusions and Future Work

6.1 Summary of Contributions

This thesis addressed the evaluation gap in procedural content generation (the trade-off between scalable automated metrics and valid human studies) through the design, deployment, and analysis of PCG Arena, a web platform for blind A/B testing of Super Mario Bros level generators.

RQ1 (Measurement). A persistent, blind, web-based arena can produce a reliable human-preference ranking. The May 2026 export contains 1,000 pairwise votes from 77 anonymous player IDs over 14 generators, yielding a stable leaderboard in which the human-authored `original` levels lead (preference score 0.833) and pattern-based generators trail. Anonymous players largely agree on which generators perform well (Section 4.3), so the aggregate ranking reflects a shared preference signal.

RQ2 (Metrics). No. Automated structural metrics act as a coarse filter, not a ranking: the four most-preferred generators have nearly indistinguishable static profiles yet span almost 0.2 in preference score, and no individual metric separates strong from weak generators (Section 4.4). By contrast, structural failure is measurable: frustrating, uncompletable levels appear consistently in gameplay trajectories and player tags (Section 4.5). A heuristic judge built on these failure indicators reproduces the coarse quality tiers in-sample ($r_s = 0.736$) and can label additional training pairs by filtering out obvious failures.

RQ3 (Generation). The collected preferences suffice to train a stronger generator: MarioDPO, fine-tuned with DPO on 473 human pairs and 2,606 judge-labelled synthetic pairs, achieves a preference score of 0.832, nearly tied with the human-authored baseline (0.833) and ahead of the next-best procedural generators, ORE (0.655) and MarioGPT (0.643).

In sum, the thesis contributes: (1) PCG Arena, a persistent open-access platform for blind pairwise evaluation of Mario level generators; (2) a user study of 1,000

blind votes across 14 generators; (3) an exploratory analysis pipeline linking votes to structure, behaviour, and tags; and (4) MarioDPO, a preference-aligned generator that approaches human-authored quality.

6.2 Limitations

The findings of this thesis should be interpreted in light of several methodological and technical limitations across the study protocol, the platform design, and the preference-learning pipeline:

- **Volunteer population and unknown player background.** Participation was voluntary and anonymous, tracked only via browser-based identifiers that can reset when cookies are cleared or devices change, and engagement was heavily concentrated in a small group of active contributors. Because no demographic or background survey was collected, we do not know participants' familiarity with platformer games or *Super Mario Bros*, nor their general gameplay preferences, making it impossible to control for individual differences in skill or taste. Moreover, the platform does not model individual evaluator reliability (e.g., via play duration or decision latency).

- **Single-attempt protocol and immediate level transition.** Each level was evaluated after a single playthrough, and the second level in a battle started immediately after the first one ended, without giving the player a pause. While this design maximised the number of pairwise comparisons collected per unit of time, a player's own performance on that single attempt (such as an early mistake) may have influenced their rating, and the immediate transition between levels left no recovery pause between evaluations.

- **Unvalidated tag taxonomy and voting UI flow.** The selectable level tags were chosen based on what seemed appropriate for this study and do not constitute a validated instrument for evaluating game levels. Furthermore, the interface did not require players to select tags, and the tag selection option disappeared immediately after a player clicked their preferred level. While this flow encouraged rapid pairwise comparisons, it discouraged thorough qualitative evaluation and contributed to tag sparsity.

- **Presence of uncompletable levels.** Because level ingestion performed only format-level validation without automated reachability checks, the evaluation pool included levels that were impossible to complete. Encountering uncompletable levels may have frustrated human testers and negatively affected their motivation.

- **Recognisability of original levels.** Players familiar with the original Nintendo levels may have recognised them despite the blind setup, introducing a

potential nostalgia or familiarity bias in favour of the human-authored baseline.

- **Dataset scale and judge bias in MarioDPO training.** The human preference dataset remains small by deep-learning standards, and the 2,606 synthetic pairs inherit the biases of J_{final}. In particular, judge features rely on short play sessions and A* agent simulations that do not fully reflect longer human exploration, and the style term privileges Nintendo-like structure, which may penalise innovative generators outside the original SMB1 distribution.

6.3 Future Directions

- **Iterative retraining.** Train a MarioDPO-v2 on the May 2026 export and study whether repeated collect–train–deploy cycles continue to improve preference scores.
- **Learned neural judge.** Replace the grid-searched J_{final} with a neural preference model trained end-to-end on the growing vote corpus.
- **Beyond Mario.** Extend the arena protocol to other content domains to test the generality of blind pairwise evaluation of procedural generators.
- **Training-aware matchmaking.** Extend AGIS to select battles that are informative both for leaderboard ranking and as DPO training pairs.

6.4 Closing Remarks

The lesson of this thesis is that human taste does not compress into a simple mathematical expression. Chapter 4 shows this empirically: the generators players prefer most have static structural profiles that are nearly indistinguishable from those of lower-ranked generators. Even the heuristic judge of Section 4.6, calibrated specifically on this dataset, primarily identifies clear structural failures.

This offers an explanation for a long-standing pattern in the field. For much of the last two decades, procedural level generation has been driven by explicit objectives: fitness functions in search-based PCG, reward functions in PCGRL, hand-tuned metrics such as leniency and linearity. Each encodes a designer's hypothesis about what makes content good, and each is falsifiable in the same way—a generator can maximise the objective and still produce levels players do not enjoy, which is precisely what Mariño et al. [12] observed and what the present data reproduces at larger scale. The difficulty was never a shortage of optimisation power; it was that the objective being optimised was a guess about human preference, not a measurement of it.

MarioDPO takes the opposite route. It never represents "fun" explicitly. It collects pairwise judgements from players and lets Direct Preference Optimisation shape the generator directly from those comparisons, using the judge only to supply broad negative coverage. The result is a generator that reaches a preference score of 0.832, statistically indistinguishable from hand-authored Nintendo levels at 0.833, and ahead of the MarioGPT checkpoint it was initialised from—improvement obtained by alignment alone, without changing the architecture or the training corpus.

Nothing in this pipeline is specific to Super Mario Bros. It requires a domain where content can be presented in pairs, a population willing to state which of two artefacts they prefer, and a generative model that can be fine-tuned on those preferences—conditions that hold for dungeons, tracks, quests, textures, and most other PCG domains. The contribution of this thesis is therefore not only a stronger Mario generator, but evidence for a general recipe: stop trying to write down what players enjoy, and start measuring it.

Statement on the Use of AI Tools

The thesis was written by the author, who is not a native speaker of English. Large Language Models, including models from OpenAI, Google, and Anthropic, were used to improve English style, clarity, and phrasing after the text had been drafted by the author. These tools were also used as support in preparing plots and figures. AI-assisted tools were also used during development as part of the author's programming workflow. The author remains responsible for the research design, implementation choices, experiments, analysis, and final content of the thesis.

Bibliography

[1] Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952. doi: 10.1093/biomet/39.3-4.324.

[2] Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Hao Zhang, Banghua Zhu, Michael Jordan, Joseph E. Gonzalez, and Ion Stoica. Chatbot arena: An open platform for evaluating LLMs by human preference. *arXiv preprint arXiv:2403.04132*, 2024. URL `https://arxiv.org/abs/2403.04132`.

[3] Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. URL `https://arxiv.org/abs/1706.03741`. arXiv:1706.03741.

[4] Steve Dahlskog and Julian Togelius. Patterns as objectives for level generation. In *Proceedings of the Second Workshop on Design Patterns in Games*, 2013. URL `http://julian.togelius.com/Dahlskog2013Patterns.pdf`.

[5] Steve Dahlskog and Julian Togelius. Procedural content generation using patterns as objectives. In *Applications of Evolutionary Computation (EvoApplications)*, volume 8602 of *Lecture Notes in Computer Science*, pages 325–336. Springer, 2014. doi: 10.1007/978-3-662-45523-4_27.

[6] Arpad E. Elo. *The Rating of Chessplayers, Past and Present*. Arco, 1978.

[7] Mark E. Glickman. Parameter estimation in large dynamic paired comparison experiments. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 48(3):377–394, 1999. doi: 10.1111/1467-9876.00120.

[8] Mark E. Glickman. Example of the Glicko-2 system. Technical report, Boston University, 2012. URL `http://www.glicko.net/glicko/glicko2.pdf`.

[9] Britton Horn, Steve Dahlskog, Noor Shaker, Gillian Smith, and Julian Togelius. A comparative evaluation of procedural level generators in the Mario AI framework. In *Proceedings of the 9th International Conference on the Foundations of Digital Games (FDG)*, 2014. URL `https://www.fdg2014.org/papers/fdg2014_paper_14.pdf`.

[10] Sergey Karakovskiy and Julian Togelius. The Mario AI benchmark and competitions. *IEEE Transactions on Computational Intelligence and AI in Games*, 4 (1):55–67, 2012. doi: 10.1109/TCIAIG.2012.2188528.

[11] Ahmed Khalifa, Philip Bontrager, Sam Earle, and Julian Togelius. PCGRL: Procedural content generation via reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, 2020. URL `https://ojs.aaai.org/index.php/AIIDE/article/view/7416`. arXiv:2001.09212.

[12] José Rafael Hernández Mariño, Walber Moreira Penna Reis, and Levi H. S. Lelis. An empirical evaluation of evaluation metrics of procedurally generated Mario levels. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, 2015. URL `https://ojs.aaai.org/index.php/AIIDE/article/view/12785`.

[13] Peter Mawhorter and Michael Mateas. Procedural level generation using occupancy-regulated extension. In *Proceedings of the IEEE Conference on Computational Intelligence and Games (CIG)*, pages 351–358. IEEE, 2010. doi: 10.1109/ITW.2010.5593333.

[14] Seong-Geon Nam, Chih-Hung Hsueh, Pasit Rerkjirattikal, and Kokolo Ikeda. Using RL to generate levels of Super Mario Bros with quality and diversity. *IEEE Transactions on Games*, 16(4):807–820, 2024.

[15] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL `https://arxiv.org/abs/2203.02155`. arXiv:2203.02155.

[16] Christopher Pedersen, Julian Togelius, and Georgios N. Yannakakis. Modeling player experience for content creation. *IEEE Transactions on Computational Intelligence and AI in Games*, 2(1):54–67, 2010.

[17] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL `https://arxiv.org/abs/2305.18290`. arXiv:2305.18290.

[18] Anurag Sarkar and Seth Cooper. Level difficulty and player skill prediction in human computation games. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, 2017. URL `https://ojs.aaai.org/index.php/AIIDE/article/view/12948`.

[19] Jacob Schrum, Olivia Kilday, Emilio Salas, Bess Hagan, and Reid Williams. Text-to-level diffusion models with various text encoders for Super Mario Bros. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, pages 110–120, 2025. doi: 10.1609/aiide.v21i1.36815. URL `https://ojs.aaai.org/index.php/AIIDE/article/view/36815`.

[20] Noor Shaker, Georgios N. Yannakakis, and Julian Togelius. Towards automatic personalized content generation for platform games. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, pages 63–68, 2010. doi: 10.1609/aiide.v6i1.12399.

[21] Noor Shaker, Julian Togelius, Georgios N. Yannakakis, Ben Weber, Tomoyuki Shimizu, Tomonori Hashiyama, Nathan Sorenson, Philippe Pasquier, Peter Mawhorter, Glen Takahashi, Gillian Smith, and Robin Baumgarten. The 2010 Mario AI championship: Level generation track. *IEEE Transactions on Computational Intelligence and AI in Games*, 3(4):332–347, 2011. doi: 10.1109/TCIAIG.2011.2166267.

[22] Noor Shaker, Georgios N. Yannakakis, and Julian Togelius. Feature analysis for modeling game content quality. In *Proceedings of the IEEE Conference on Computational Intelligence and Games (CIG)*, 2011. doi: 10.1109/CIG.2011.6031998.

[23] Noor Shaker, Miguel Nicolau, Georgios N. Yannakakis, Julian Togelius, and Michael O'Neill. Evolving levels for Super Mario Bros using grammatical evolution. In *Proceedings of the IEEE Conference on Computational Intelligence and Games (CIG)*, pages 304–311, 2012. doi: 10.1109/CIG.2012.6374170.

[24] Noor Shaker, Julian Togelius, and Mark J. Nelson. *Procedural Content Generation in Games: A Textbook and an Overview of Current Research*. Springer, 2016. doi: 10.1007/978-3-319-42716-4. URL `https://www.pcgbook.com`.

[25] Gillian Smith and Jim Whitehead. Analyzing the expressive range of a level generator. In *Proceedings of the 2010 Workshop on Procedural Content Generation in Games (PCGames)*, pages 4:1–4:7. ACM, 2010. doi: 10.1145/1814256.1814260.

[26] Rion Snow, Brendan O'Connor, Daniel Jurafsky, and Andrew Y. Ng. Cheap and fast — but is it good? Evaluating non-expert annotations for natural language tasks. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 254–263. Association for Computational Linguistics, 2008. URL `https://aclanthology.org/D08-1027/`.

[27] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel M. Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F. Christiano. Learning

to summarize with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. URL https://arxiv.org/abs/2009.01325. arXiv:2009.01325.

[28] Shyam Sudhakaran, Miguel González-Duque, Claire Glanois, Matthias Freiberger, Elias Najarro, and Sebastian Risi. MarioGPT: Open-ended text2level generation through large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://papers.neurips.cc/paper_files/paper/2023/file/a9bbeb2858dfbdbd4c19814e5d80ec60-Paper-Conference.pdf. arXiv:2302.05981.

[29] Adam Summerville and Michael Mateas. Super Mario as a string: Platformer level generation via LSTMs. In *Proceedings of the 1st International Joint Conference of DiGRA and FDG*, 2016. doi: 10.26503/dl.v2016i1.752. arXiv:1603.00930.

[30] Adam Summerville, José Rafael Hernández Mariño, Sam Snodgrass, Santiago Ontañón, and Levi H. S. Lelis. Understanding Mario: An evaluation of design metrics for platformers. In *Proceedings of the 12th International Conference on the Foundations of Digital Games (FDG)*, 2017. doi: 10.1145/3102071.3102080.

[31] Adam Summerville, Sam Snodgrass, Matthew Guzdial, Christoffer Holmgård, Amy K. Hoover, Aaron Isaksen, Andy Nealen, and Julian Togelius. Procedural content generation via machine learning (PCGML). *IEEE Transactions on Games*, 10(3):257–270, 2018. doi: 10.1109/TG.2018.2846639.

[32] Julian Togelius, Georgios N. Yannakakis, Kenneth O. Stanley, and Cameron Browne. Search-based procedural content generation: A taxonomy and survey. *IEEE Transactions on Computational Intelligence and AI in Games*, 3(3):172–186, 2011. doi: 10.1109/TCIAIG.2011.2148116.

[33] Vanessa Volz, Jacob Schrum, Jialin Liu, Simon M. Lucas, Adam Smith, and Sebastian Risi. Evolving Mario levels in the latent space of a deep convolutional generative adversarial network. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, pages 221–228. ACM, 2018. doi: 10.1145/3205455.3205517.

[34] Georgios N. Yannakakis and Julian Togelius. Experience-driven procedural content generation. *IEEE Transactions on Affective Computing*, 2(3):147–161, 2011. doi: 10.1109/T-AFFC.2011.6.

[35] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-Judge with

MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL `https://arxiv.org/abs/2306.05685`. arXiv:2306.05685.

[36] Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019. URL `https://arxiv.org/abs/1909.08593`.